学校代码：10286
分 类 号：TP118
密　　级：公开
U D C ：004.8
学　　号：184563

東南大學

SOUTHEAST UNIVERSITY

硕士学位论文

基于知识图谱的 AIOps 根因分析系统的设计与实现

（学位论文形式：应用研究）

研究生姓名：罗安源

导 师 姓 名：漆桂林 教授

丁岩　高工

申请学位类别　工 程 硕 士　　学位授予单位　东 南 大 学

工程领域名称　计算机技术　　论文答辩日期　2021 年　6 月 1 日

研究方向　知识图谱，网络空间安全　　学位授予日期　2021 年　6 月　日

答辩委员会主席　倪巍伟 教授　　评　阅　人　翟玉庆　教授

唐彦 副教授

2021 年 6 月 1 日

中国知网 https://www.cnki.net

東南大學

硕士学位论文

基于知识图谱的 AIOps 根因分析系统的设计与实现

专 业 名 称 ： 计算机技术

研究生姓名 ： 罗安源

导 师 姓 名 ： 漆桂林

校外导师姓名： 丁 岩

中国知网 https://www.cnki.net

DESIGN AND IMPLEMENTATION OF AIOPS ROOT CAUSE ANALYSIS SYSTEM BASED ON KNOWLEDGE GRAPH

A Thesis Submitted to

Southeast University

For the Professional Degree of Master of Engineering

BY

LUO An-yuan

Supervised by

Prof. QI Gui-lin

and

S.E DING Yan

School of Cyber Science and Engineering

Southeast University

May 2021

中国知网 https://www.cnki.net

摘 要

随着虚拟化技术的快速发展，分布式系统越来越庞大且复杂。当系统出现故障时，运维人员很难在短时间内找到根本原因，系统将处于不稳定状态。传统运维越来越困难，智能运维(Artificial Intelligence for IT Operations , AIOps）被提上日程。及时找出故障的根因，保证系统的安全稳定运行是智能运维最基础的功能。因此，研究分布式系统的根因分析方法，找出导致系统故障的根本原因，保证系统安全运行变得非常重要。

近年来，越来越多的研究者开始关注这些问题，基于依赖图的方法和基于因果图的方法在应对复杂系统故障根因分析问题上取得了一些不错的成果。但是这些方法均存在两点局限性：1、它们尚未研究如何利用历史数据中隐藏的显式知识来指导当前的根因分析；2、它们无法完整地解释出事件层次的故障触发路径。基于此，本文针对分布式系统中的故障为每类故障构建了故障知识图谱，提出了一种结合图卷积网络和知识图谱的故障根因分析方法，设计并实现了一个基于知识图谱的根因分析系统。本文的主要贡献包括：

1) 针对分布式系统中的故障为每类故障构建了故障知识图谱。本文通过从分布式系统历史故障数据中挖掘事件之间的关系，为每个故障构建故障传播图，然后利用合并算法来获得相似故障传播图的相似结构，以此来构建每类故障的故障知识图谱，最后进行了相关实验，测试并评估了故障知识图谱的构建方法。
2) 提出了一种结合图卷积网络和知识图谱的故障根因分析方法。本文通过实时构建在线故障传播图，并使用图卷积神经网络比较在线故障传播图与故障知识图谱的相似度，以确定实际故障类型，最后利用故障与抽象根因事件之间的对应关系，通过排名策略来定位具体的根因事件。本文在阿里云平台上的分布式系统上进行了实验，设置了多组对比实验来验证本文方法的有效性，实验结果表明，本文提出的方法在精确率上优于其他的根因分析方法。
3) 设计并实现了一个基于知识图谱的根因分析系统。本文基于上述提出的方法设计并实现了一个故障根因分析系统，经过测试，系统满足各项功能和性能需求。

综上所述，本文针对智能运维中故障根因分析问题，构建了故障知识图谱，并提出了一种基于知识图谱的故障根因分析方法，最终实现了相应的系统。相比已有系统，该系统能进一步提高故障根因分析的精确率。

关键词：根因分析；知识图谱；智能运维；图卷积神经网络

中国知网 https://www.cnki.net

Abstract

With the rapid development of virtualization technology, distributed systems are becoming larger and more complex. When the system fails, it is difficult for the on-call personnel to find the root cause in a short time, and the system will be in an unstable state. Traditional operation and maintenance is becoming more and more difficult, and artificial intelligence for IT operations (AIOps) is put on the agenda. Finding out the root cause of faults in time and ensuring the safe and stable operation of the system are the most basic functions of AIOps. Therefore, it is very important to study the root cause analysis method of distributed system, find out the root cause of system failure and ensure the safe operation of the system.

Recently, more and more researchers have begun to pay attention to these problems. The methods based on dependency graph and causality graph have achieved some good results in dealing with the problem of fault root cause analysis of complex systems. These methods have two limitations: 1. They have not studied how to use explicit knowledge hidden in historical data to guide current root cause analysis; 2. They cannot completely explain the fault triggering path at the event level. Based on this, this thesis constructs a fault knowledge graph for each type of fault, proposes a fault root cause analysis method combining graph convolution network and knowledge graph, and designs and implements a root cause analysis system based on knowledge graph. The main contributions of this thesis include:

1) This thesis constructs a fault knowledge graph for each type of fault. In this thesis, the relationship between events is mined from the historical fault data of distributed system, and the fault propagation graph is constructed for each fault. Then, the similar structure of the similar fault propagation graph is obtained by using the merging algorithm, so as to construct the fault knowledge graph of each type of fault. Finally, related experiments are carried out to test and evaluate the construction method of fault knowledge graph.
2) This thesis proposes a fault root cause analysis method combining graph convolution network and knowledge graph. In this thesis, the online fault propagation graph is constructed in real time, and the similarity between online fault propagation graph and fault knowledge graph is compared by using graph convolution neural network to determine the actual fault type. Finally, the corresponding relationship between fault and abstract root cause event is used to locate specific root cause events by ranking strategy. In this thesis, experiments are carried out on the distributed system on Alibaba Cloud platform, and several groups of comparative experiments are set up to verify the effectiveness of the proposed method. The experimental results show that the proposed method is superior to other root cause analysis methods in precision.

中国知网 https://www.cnki.net

3) This thesis designs and implements a root cause analysis system based on knowledge graph. In this thesis, a fault root cause analysis system is designed and implemented based on the above-mentioned methods. After testing, the system meets various functional and performance requirements.

In summary, aiming at the problem of fault root cause analysis in AIOps, this thesis constructs a fault knowledge graph, proposes a fault root cause analysis method based on knowledge graph, and finally implements the corresponding system. Compared with existing systems, this system can further improve the precision of fault root cause analysis.

Key words: Root Cause Analysis; Knowledge Graph; Graph Convolution Network; AIOps

中国知网 https://www.cnki.net

目 录

摘 要.....I
Abstract.....II
目 录.....IV
插图目录.....VI
表格目录.....VII
缩略词表.....VIII
第一章 绪论.....1
1.1 研究背景与意义.....1
1.2 国内外研究现状.....3
1.2.1 早期的根因分析方法.....3
1.2.2 基于因果图的方法.....4
1.2.3 基于依赖图的方法.....5
1.2.4 基于显式知识存储的方法.....5
1.2.5 研究现状小结.....6
1.3 研究目标.....6
1.4 研究内容.....6
1.5 论文结构.....7
第二章 背景知识.....9
2.1 AIOps 和故障根因分析.....9
2.1.1 AIOps.....9
2.1.2 故障根因分析.....9
2.2 知识图谱.....10
2.3 图卷积神经网络.....11
2.4 聚类算法与图核.....13
2.4.1 聚类算法.....13
2.4.2 图核.....14
2.5 本章小结.....15
第三章 故障知识图谱构建.....17
3.1 相关定义.....17
3.2 事件生成.....19
3.3 故障传播图构建.....19
3.4 故障知识图谱构建方法.....23
3.4.1 故障知识沉淀.....23
3.4.2 故障知识标注.....25
3.5 实验.....26
3.5.1 数据集.....26
3.5.2 评测指标.....27
3.5.3 实验结果.....27
3.6 本章小结.....30

中国知网 https://www.cnki.net

第四章 基于图卷积神经网络的故障根因分析.....31
4.1 故障根因分析方法概览.....31
4.2 图相似度计算方法.....31
4.3 基于图卷积神经网络的故障根因分析方法.....33
4.4 RCA 案例分析.....34
4.5 实验.....35
4.5.1 数据集与评测指标.....35
4.5.2 实验结果.....35
4.6 本章小结.....36
第五章 故障根因分析系统的设计与实现.....37
5.1 故障根因分析系统需求分析.....37
5.1.1 功能需求.....37
5.1.2 非功能需求.....38
5.2 系统总体设计.....39
5.2.1 设备图谱构建模块概要设计.....41
5.2.2 异常发现模块概要设计.....41
5.2.3 故障知识图谱构建模块概要设计.....41
5.2.4 根因分析模块概要设计.....41
5.3 系统详细设计与实现.....42
5.3.1 设备图谱构建模块详细设计与实现.....42
5.3.2 异常发现模块详细设计与实现.....44
5.3.2 故障知识图谱构建模块详细设计与实现.....45
5.3.3 根因分析模块详细设计与实现.....46
5.4 系统测试.....47
5.4.1 系统测试环境.....47
5.4.2 设备查询测试.....47
5.4.3 故障根因分析测试.....49
5.4.4 系统性能测试.....50
5.5 本章小结.....50
第六章 总结与展望.....51
6.1 本文工作总结.....51
6.2 未来工作展望.....51
致 谢.....52
参考文献.....53
作者攻读硕士学位期间科研成果.....57

中国知网 https://www.cnki.net

插图目录

图 1-1 火车票预定系统的分布式架构示例 1
图 1-2 故障“访问失败，返回 503 错误”的触发方式示例 3
图 1-3 论文结构图 7

图 2-1 故障根因分析工作流程 10
图 2-2 GCN 的基本框架图 12
图 2-3 R-GCN 模型示意图 13

图 3-1 部分故障传播图示例 18
图 3-2 事件生成方法示意图 19
图 3-3 故障知识图构建示意图 23
图 3-4 h-聚类精确率折线图 29
图 3-5 M-FPG_F1 折线图 29
图 3-6 θ-FPG_F1 折线图 29

图 4-1 故障根因分析方案概览 31
图 4-2 图相似度计算模型 32
图 4-3 根因分析方法 33
图 4-4 故障根因分析实例 34

图 5-1 系统用例图 37
图 5-2 故障根因分析系统框图 39
图 5-3 部分设备本体结构 42
图 5-4 查询某类设备及其周边节点 43
图 5-5 某服务器节点的生命周期 43
图 5-6 在设备图谱中查询设备的监控信息 44
图 5-7 日志数据异常检测流程图（左）和 metric 数据异常检测流程图（右） 44
图 5-8 故障知识图谱构建与更新流程图 45
图 5-9 故障根因分析流程图 46
图 5-10 模糊查询特定属性的节点 48
图 5-11 查询 vpc 节点的监控信息 49
图 5-12 f3 故障的故障根因分析结果及其触发路径展示 50

中国知网 https://www.cnki.net

表格目录

表 3-1 运维事件分类表 18
表 3-2 数据集详细信息表 26
表 3-3 因果关系标注 27
表 3-4 关系分类器评估表 28
表 3-5 故障知识图谱构建结果 28

表 4-1 RCA 对比实验结果 36

表 5-1 设备图谱本体统计 42
表 5-2 系统测试实验环境 47
表 5-3 设备查询功能测试用例 48
表 5-4 故障根因分析功能测试用例 49
表 5-5 系统性能测试结果 50

中国知网 https://www.cnki.net

缩略词表

缩略词	英文全称	中文解释
AIOps	Artificial Intelligence for IT Operations	智能运维
RCA	Root Cause Analysis	根因分析
FPG	Fault Propagation Graph	故障传播图
FKG	Fault Knowledge Graph	故障知识图谱
KG	Knowledge Graph	知识图谱
R-GCN	Relational Graph Convolutional Network	关系图卷积网络

中国知网 https://www.cnki.net

第一章 绪论

本章首先从故障根因分析的重要性、困难性和当前根因分析的问题三个方面介绍本文的研究背景，接着介绍本文课题的研究现状，然后阐述本文的研究内容与研究目标，最后说明本文的结构安排。

1.1 研究背景与意义

随着互联网的飞速发展，容器技术也随之兴起。基于容器的分布式系统管理系统Kubernetes[1]通过网络将各个单独的服务器连接起来，分布式系统中的容器也由网络相互连接，使得数据可以在容器和容器之间、容器和服务器之间、服务器和服务器之间通过网络进行相互传输。然而，如果其中某一个容器出现故障，那么故障可能会通过网络影响其他容器，甚至在某种极端情况下可能会影响整个分布式系统。目前，许多大型企业都已经将核心的系统转移到分布式系统中，一些政府也将本地的应用转向虚拟化技术与容器，因此，提升系统的安全性则变得相当重要。可是，无论如何提升系统的安全性能，都不可能百分之百的避免意外网络故障和系统故障的发生。因此，当发生紧急故障时，在庞大的分布式系统中快速定位网络故障和系统故障，及时解决故障，防止故障造成更大的损失，对于保证企业和政府系统的安全至关重要。

很多企业软件和系统都是建立在一个庞大、复杂、跨协议层的分布式环境之上的。这种分布式环境的技术、软件、配置通常会不断地演变；其软硬件也难以避免发生故障与变更；网络流量会发生不可预知的变化，甚至会发生网络安全攻击事件等等。部署在这种复杂架构下的网络应用出现故障或者遭受网络攻击时，故障检测和故障根因分析就显得异常困难。

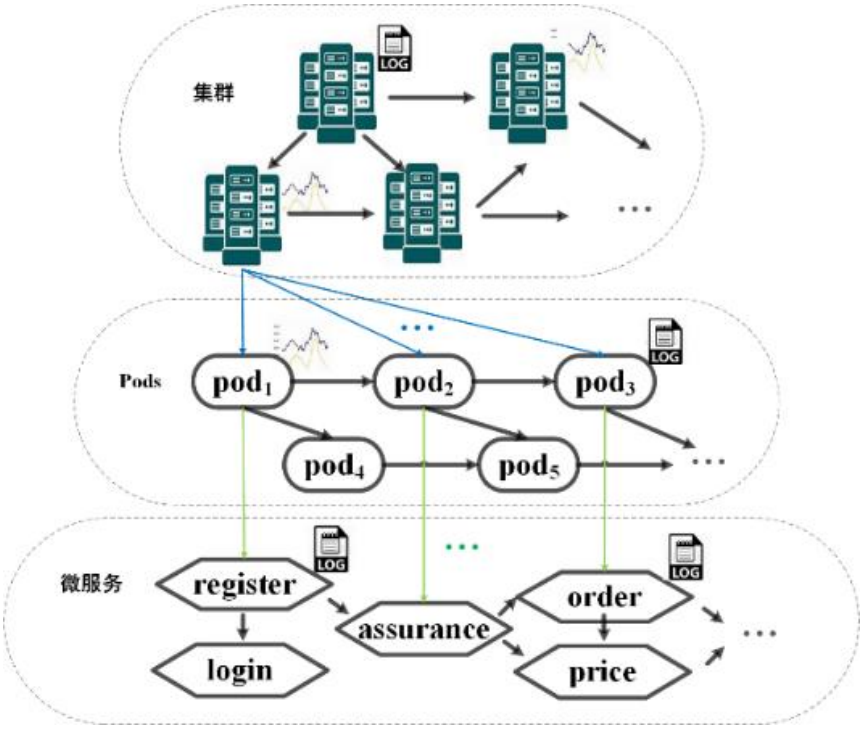

图 1-1 火车票预定系统的分布式架构示例

中国知网 https://www.cnki.net

尽管，目前的各类监控工具使得分布式系统运行状态的可见度有了较大幅度的提升，但是当它们遇到网络故障或系统故障时，面对海量的网络监控数据和庞大的系统，仅依赖网络运维人员在高压下人力做出迅速、准确的决策仍比较困难的。具体难点如下：

1) 分布式系统的网络拓扑结构复杂：由于网络应用部署在规模庞大的分布式系统上，物理机和虚拟机的网络拓扑结构复杂，导致故障根因分析异常困难。例如，图 1-1 显示了一个实际的分布式系统的结构，从图中可以看到三种组件(服务器群集，Pod 和微服务)之间存在复杂的依赖关系(用箭头表示)。
2) 分布式系统故障修复时间紧迫：为了监视所有组件的状态，指标监控记录(Metric，如内存使用率记录)和日志的数量将急剧增加，当故障发生，运维人员需要扫描数百个不同的指标仪表板和报警来寻找故障根因，此过程非常耗时，在 500 台服务器规模的分布式系统上，平均需要 2 小时才能完全定位到一个故障。如今网络已经渗透到人们的日常生活中，网络服务系统出现短暂故障不仅会影响人们的生活还会给社会带来巨大的损失，故障修复时间越久意味着故障带来的损失越大。例如，Amazon.com 在 2013 年的 45 分钟停机故障造成了 500 万美元损失[2]，同时影响了数百万人的工作和生活。
3) 高水平互联网运维人员稀缺：由于网络应用部署在分布式系统上，网络运维人员不仅要熟悉各个微服务之间的调用方式，还需要对分布式系统的网络拓扑结构和配置有一定的了解，这样的高水平互联网运维人才是很稀缺的。

基于上述难点，传统运维很难在日益复杂的系统中快速准确地发现故障根本原因，来达到保证系统安全性的目的，因此智能运维被提上日程。智能运维是将人工智能的能力与运维相结合，通过机器学习的方法来提升运维效率的一种运维新方式。及时找出故障的根因，保证系统的安全稳定地运行是智能运维最基础的功能。智能运维解决上述问题的思路就是逐渐减少人力在决策中所占的比例，逐渐增加人工智能技术在决策中的比例，让故障检测和故障处理越来越趋近智能化。为了尽可能地提升故障检测的效率和效果，解决不安全隐患，亟需将领域知识与动态决策引入故障根因分析中。如何自动地为分布式系统执行高质量的根因分析，找出导致系统故障的根本原因是智能运维的重要问题。尽管根因分析存在许多相关工作[3]如：基于概率统计[11, 12]的方法、基于依赖图[6, 10]的方法和基于因果图[7]的方法等，但它们存在以下两点局限：

局限 1. 现有方法忽略了历史数据中的显式知识或模式。历史数据是现有方法中根因分析的重要挖掘来源[4, 5]，他们使用统计模型来检测异常或分析数据相关性，但忽略了历史数据中的显式知识或模式。这些显式知识或模式可以很好地指导故障根因分析。例如，假设对故障“访问失败，返回 503 错误”(参见图 1-2)有一个概括的触发模式，当再次发生此故障时，原因可能是“内存已满”。

局限 2. 现有方法无法完整地解释出事件层次的故障触发路径。故障的传播路径包含着故障的传播信息，有效利用故障传播路径可以提升根因分析的精度。最新工作虽然引入了系统

中国知网 https://www.cnki.net

图[6]、因果图[7]描绘故障的传播来辅助根因分析，但是系统图无法展示故障在设备内部的传播情况，而因果图只靠因果关系不能完整地描绘故障的传播。

基于以上研究背景与已有方法的局限性，本文从 Kubernetes 分布式系统故障入手，提出一种基于知识图谱的故障分析方法，根据该方法实现了一个故障根因分析系统，辅助网络运维人员进行线上网络故障诊断和异常排查分析，以此来逐步减少人力在决策中所占的比例，提高故障检测的效率与云计算场景下网络应用的安全性。

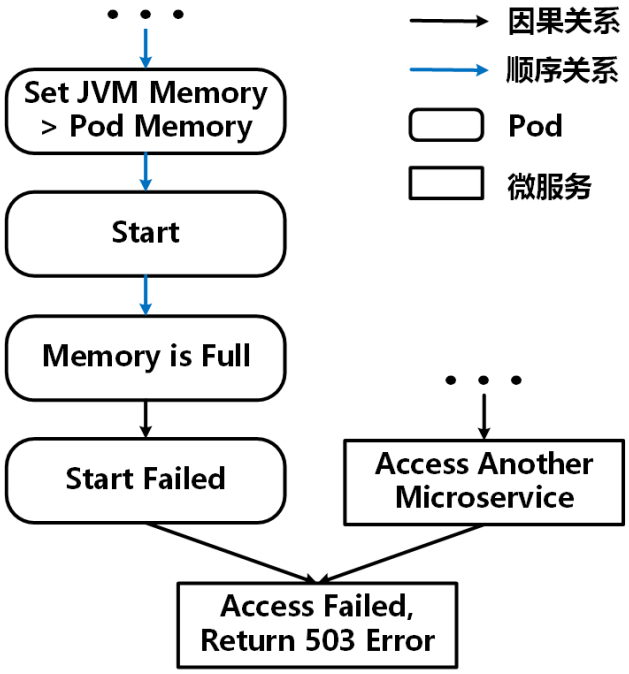

图 1-2 故障“访问失败，返回 503 错误”的触发方式示例

1.2 国内外研究现状

根因分析(Root Cause Analysis, RCA)工作又叫根因溯源或根因定位。本节首先介绍一些早期的根因分析方法，然后介绍一些最新的根因分析方法，最后分析现有方法的局限性。

1.2.1 早期的根因分析方法

早期的根因分析工作通过建立根因分析模型并在模型上推理得到根因。模型主要可以分为两类：概率模型和确定性模型。

在确定性模型中，模型中表达的已知事实或推论没有不确定性。确定性模型如规则模型、Codebook、Fault Tree 和一些机器学习模型等。基于规则的方法[8, 9]是存储和管理知识的有效方法。规则由相关领域专家构建，是一种质量很高的结构化知识。规则构建完成后，可以利用它进行一些故障推理。早期系统结构比较简单，使用规则可以完成故障根因分析任务，但是随着系统复杂性的提高，规则的规模也不断庞大，构建规则的成本也越来越高，而且规则的迁移性比较弱，它在预定义的场景中表现良好，但是在拓扑结构频繁变化的场景中，规则的适用性将变得很差。并且它缺乏从经验中学习的能力，不能处理看不见的问题，自动更新

中国知网 https://www.cnki.net

系统知识。Codebook[4]是一种典型的基于编码的查找根本原因的方法。在这种方法中，事件被分解为症状和问题，并根据历史数据自动构建它们之间的关联图。对于每个问题，编码由 0 和 1 组成的序列表示，其中 1 表示特定症状的发生，0 表示未观察到症状。然后利用编码理论在观察到的症状中识别出根本原因事件。机器学习也是故障根因分析的一个重要方向，包括逻辑回归模型[18]、决策树[19]、k 近邻等监督模型[20]，聚类等无监督模型[21]，人工神经网络等深度学习模型[22]。机器学习的根因分析方法通常从系统数据中提取特征，描述系统的当前状态，并根据这些特征训练分类器进行故障分类。他们有很强的学习能力，能够抵抗噪音和不一致。然而，这些模型的可解释性不高。还有一些综合性的工作，Wang[23]提出一种方法，该方法首先使用 DeepLog[24]给日志进行异常打分，经过数据扩充后和 Metric 数据进行相关性分析，然后得出可能引发异常日志的 Metric 的一个排名。这种方法虽然将 Metric 和日志联系起来，但是忽视了日志间的故障传播。

概率模型能够处理已知事实或推论中的不确定性。概率模型如贝叶斯网络、隐马尔可夫模型、概率逻辑等。基于概率的模型[11, 12]是根因分析的经典模型，能够自动、动态地进行分析。主要的概率模型有概率逻辑[13]、贝叶斯网络[14]、马尔可夫逻辑网络[15, 16]等。基于概率模型的方法是非确定性方法，相对于确定性方法它引入了概率的不确定性，避免了非正即负，更加的灵活。基于概率模型的故障根因分析方案，一般将故障作为随机变量，然后通过建立概率模型计算相互关联变量之间的转移概率，可以分析分析变量之间的依赖关系或者实现因果推理，这种模型具有非常好的可解释性，与神经网络之类的黑盒模型相比，更符合人类的思维习惯。贝叶斯网络是一种概率模型，在贝叶斯网络中，节点表示随机变量，有向边表示相互关联变量之间的转移概率。但由于贝叶斯推理是 NP 难问题[17]，在实际应用中很难实现。

直接使用这些模型进行根因分析在早期系统结构比较简单的时候可以发挥不错的效果。但是随着系统结构越来越庞大，故障的传播路径也越来越复杂。故障的传播路径蕴含着很多故障信息，将故障传播路径这种显式知识引入故障根因分析不仅可以提高根因分析精确率，还可以帮助运维人员了解故障传播过程。然而早期模型没有利用到故障的传播路径、系统的拓扑结构这些显式的知识，直接使用这些模型进行根因分析的效果越来越差。最新的工作将故障传播图引入故障根因分析，主要有基于因果图的方法和基于依赖图的方法。

1.2.2 基于因果图的方法

基于因果图的方法是在根因分析过程中加入了事件层次的故障传播路径，并且传播图中的关系只有因果关系。

Nie(2016)[7]提出了一种基于因果图的自动诊断系统，以帮助运维人员找到故障根本原因。因果图主要从监控系统的历史数据中提取。该自动诊断系统首先利用数据挖掘方法提取初始因果图，一旦发生故障，它会基于因果关系图使用排名算法列出前 k 个故障根因，然后领域专家检查这 k 个故障根因，给它们贴上正确或错误的标签，最后自动诊断系统使用监督学习算法将标签作为输入来调整因果图，以迭代地提高诊断精度。虽然该方法既不需要关于基于 web 的服务的设计和实现细节的知识，也不需要检测服务的源代码。然而，这种方法要求因

中国知网 https://www.cnki.net

果图的高精度。一旦因果图的准确性不高，根因分析结果就会直线下降。而且因果图的方法虽然可以描绘事件的传播，但是因果图中只有因果关系，而只靠因果关系不能完整地描绘故障的传播。

1.2.3 基于依赖图的方法

基于依赖传播图的方法是在根因分析过程中加入了故障在依赖图传播的路径，依赖图可以是设备依赖图[6, 10]、微服务依赖图、Metric 依赖图[5]等，传播图中的关系是各节点的依赖关系。

Brandón (2120)[6]提出了一种基于设备依赖图的方法。该方法构造了包含异常设备子图并标注根源节点的模式库。模式库中的每个异常设备子图代表一种故障。当系统发生故障时，将当前系统的异常设备子图与模式库中的每个异常设备子图进行比较，找出最相似的异常设备子图的根本原因作为当前故障的根本原因。这种方法很难确定异常设备子图的尺度。如果异常设备子图太小，则根本原因可能不在该子图的范围内。如果异常设备子图太大，异常设备子图可能包含许多不必要的设备节点，这些节点很可能在相似性计算中成为噪声。同时，如果返回给用户的异常设备子图过大，用户很难分析异常设备子图。Wang(2019)[10]通过在系统组件图上运行故障传播算法来进行根因分析。虽然这种方法可以返回与故障相关的组件和组件上的警报，但它不考虑警报顺序。由于没有时间维信息，这种方法无法准确确定报警之间的触发关系。Meng(2020)[5]基于 Metric 依赖图提出了根因分析系统 MicroCause，该系统首先使用改进的 PC 算法从 Metric 中发现各种指标之间的因果关系，生成因果关系图，然后使用异常检测算法找到 Metric 中异常，并使用面向时间的随机游动方法在因果图中进行随机游走，从而找出 TopN 的根因。

基于设备依赖图和基于微服务依赖图的方法虽然利用了故障在设备和微服务上的传播路径，但是忽略了故障在节点内部的传播情况。基于 Metric 依赖图的方法只考虑到了故障在不同 Metric 之间传播，忽略了故障在同一 Metric 的传播。现有的基于依赖图的方法使用的故障传播路径都不完整。

1.2.4 基于显式知识存储的方法

按照是否引入显式知识，根因分析方法可分为基于显式知识的方法和基于统计模型的方法。历史数据是现有方法中自动根因分析的重要挖掘来源，基于统计模型使用统计模型来检测异常或分析数据相关性，但这也忽略了历史数据中可以很好地辅助根因分析的明确知识或模式。

Brandón (2120)[6]提出了一种基于显式知识存储的方法。该方法构造了包含异常设备子图并标注根源节点的模式库。模式库中的每个异常设备子图代表一种故障。当系统发生故障时，将当前系统的异常设备子图与模式库中的每个异常设备子图进行比较，找出最相似的异常设备子图的根本原因作为当前故障的根本原因。但是此方法忽略了设备内部显式的故障传播知识。

中国知网 https://www.cnki.net

少数现有工作利用了显式方式存储知识，但是存储知识的可解释性不足，对知识的挖掘不充分。

1.2.5 研究现状小结

早期的故障根因分析工作忽略了故障传播路径这个重要的信息，基于因果图和基于依赖图的方法虽然引入了故障传播路径，但是因果图中只有因果关系，只靠因果关系不能完整地描绘故障的传播，而现有基于依赖图的方法使用的故障传播路径都不完整。而且现有工作大部分忽略了历史数据中的明确知识或模式，少数现有工作利用了显式方式存储知识，但是存储知识的可解释性不足，对知识的挖掘不充分。

1.3 研究目标

旨在提出一种故障根因分析方法，设计并实现一个故障根因分析系统。该方法试图采用构建故障知识图谱的方法完善当前故障根因分析方法的不足。本文试图首先通过对分布式系统中的故障进行建模构建故障知识图谱，然后使用深度神经网络结合故障知识图谱实现故障根因分析，最终将上述研究方法应用在故障根因分析系统上，利用故障根因分析结果辅助运维人员解决分布式系统中出现的故障，提高整个分布式系统系统的容错性，从而提升分布式系统的安全性能。

1.4 研究内容

本文的研究主要包括以下三个部分：

(1) 研究故障知识图谱构建技术

故障知识图谱可以描述分布式系统中故障与故障之间的引发关系，如果可以将某类故障出现时周边设备的状态和系统事件与故障之间的关系沉淀为知识存放在故障知识图谱中，那么对于新出现的故障，只需要查看某些设备的状态和其他出现的一些事件，就可以定位到故障的根本原因，因此构建一张故障知识图谱是很重要的。但是由于故障的种类繁多，故障细粒度难以控制，故障事件与正常事件的范围的难以界定，故障与故障之间的关系难以判断，故障知识难以沉淀，需要有一整套流程解决这些问题。本文旨在研究一套故障知识图谱构建框架，基于这套框架可以生成事件、挖掘事件之间的关系并构建故障知识图谱。

(2) 研究基于图卷积网络的故障根因分析方法

在得到故障知识图谱的基础上，如何利用故障知识图谱中事件之间的关系来进行故障根因分析是本文研究的重点。本文通过构建一个在线故障传播图，并使用图卷积神经网络比较每个故障传播图与故障知识图谱之间的相似性，以确定实际故障类型，最后利用故障和抽象的根因事件之间的对应关系，通过排序策略来定位故障的根因事件。

中国知网 https://www.cnki.net

(3) 设计并实现故障根因分析系统

基于以上的研究，本文设计了一个具有分布式系统故障根因分析功能的系统，并完成该系统的开发。系统可以根据故障类型和周边设备拓扑结构搜寻故障根本原因，根据分析结果从设备层和事件层可视化分布式系统故障的传播路径，辅助运维人员解决分布式系统故障。

1.5 论文结构

论文结构图如图 1-3。本文共包含六章，具体内容安排如下：

第一章介绍本文的研究背景和相关工作研究现状，并阐述主要的研究目标和内容。

第二章介绍与本文相关的背景知识，主要包括知识图谱、智能运维、故障根因分析以及接下来的章节涉及的技术知识。

第三章介绍故障知识图谱的构建框架，包括事件生成、故障传播图构建和故障知识图谱的沉淀。最后进行相关实验，测试并评估知识图谱的构建方法。本章构建的故障知识图谱将用于第四章的故障根因分析。

第四章介绍基于故障知识图谱的故障根因分析框架，包括图相似度计算模型和根因分析方法。最后进行相关实验，并对实验结果进行评估和分析。

第五章基于第三章和第四章提出的故障根因分析方法设计并实现一个故障根因分析系统。

第六章总结全文的工作并对未来的工作进行展望。

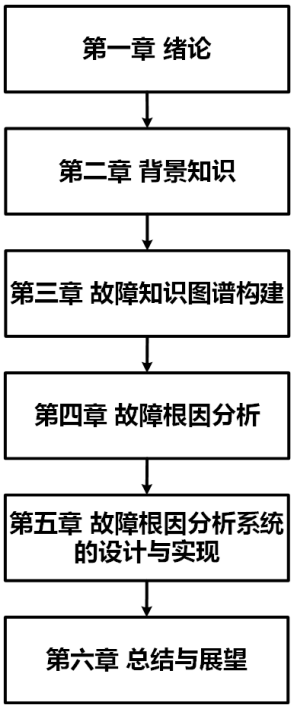

图 1-3 论文结构图

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

第二章 背景知识

本章首先介绍了智能运维和故障根因分析，然后介绍了知识图谱的相关背景。最后介绍本文中故障根因分析涉及的技术知识，包括图相似性度量的方法、图卷积神经网络、聚类算法和图核。

2.1 AIOps 和故障根因分析

2.1.1 AIOps

大型计算机系统(例如云、物联网和嵌入式环境)是一项关键技术，它改变了包括医疗保健、金融，制造业、教育和交通。数十亿的设备和用户进行通信依靠于这些系统的可靠性和可用性。大型计算机系统越来越多地使用分布式技术，这种技术提供了更大的系统灵活性，但同时大大增加了它们的复杂性。由于软件和硬件的复杂性和不可避免的弱点，这些系统很容易出现故障。几项研究表明，这种故障会导致可靠性降低、财务成本高，并会影响关键应用[35]。因此，任何系统或基础设施都不允许失去控制，因为服务质量非常重要。

大型服务提供商意识到对具有高可用性的持续服务的需求，因此已经部署了许多措施，例如站点可靠性工程师和 DevOps。但是，计算机系统的规模和复杂性已稳定地增加到手动操作已不可行的水平。操作员开始使用人工智能工具来自动化各种操作任务系统监视，异常检测，根本溯源和恢复。

智能运维(Artificial Intelligence for IT Operations , AIOps)是一个横跨机器学习，大数据，流分析和 IT 运营管理的一个新兴领域。作为一个领域，AIOps 是为 IT 运营管理设定未来标准的最佳选择。为此，AIOps 存在几个挑战。首先，它需要结合其他独立研究部门的研究领域，例如软件可靠性工程。其次，它需要新颖的建模技巧并且需要了解不同系统的动态。此外，它还需要及早发现新出现的问题，及时的补救措施。

故障根因分析是 AIOps 领域的一项基础研究，是保证高可用性服务的重中之重，是维持系统安全稳定的必要步骤。

2.1.2 故障根因分析

AIOps 领域中的故障根因分析是指在大型计算机系统(例如云、物联网和嵌入式环境)中对出现的故障进行检测，自动分析引发故障的根本原因。它的困难之处在于系统结构复杂、故障数据庞大和大量多源异构数据等。

如图 2-1 所示，AIOps 领域的故障根因分析技术大多数共享一个基本的工作流程。首先结合领域知识、系统知识和被诊断系统的故障构建一个根因分析模型。这些类型的信息并非都必须使用，但是也有考虑所有这些信息的方法。然后将诊断系统的故障输入到根因分析模型所使用的算法和模型中进行推理，最终得到故障的根本原因或者故障根本原因的解释。当系统发生变化(即系统知识发生变化)时，例如添加/删除组件或组件之间的连接，模型可能会

中国知网 https://www.cnki.net

进行更新。当发生这种变化时，有些模型可以增量更新，而有的模型则需要重建模型。领域知识的变化一般比系统知识的变化要少得多，而且一旦领域知识改变，那么整个模型可能需要重构，因此这里没有明确考虑领域知识更新。

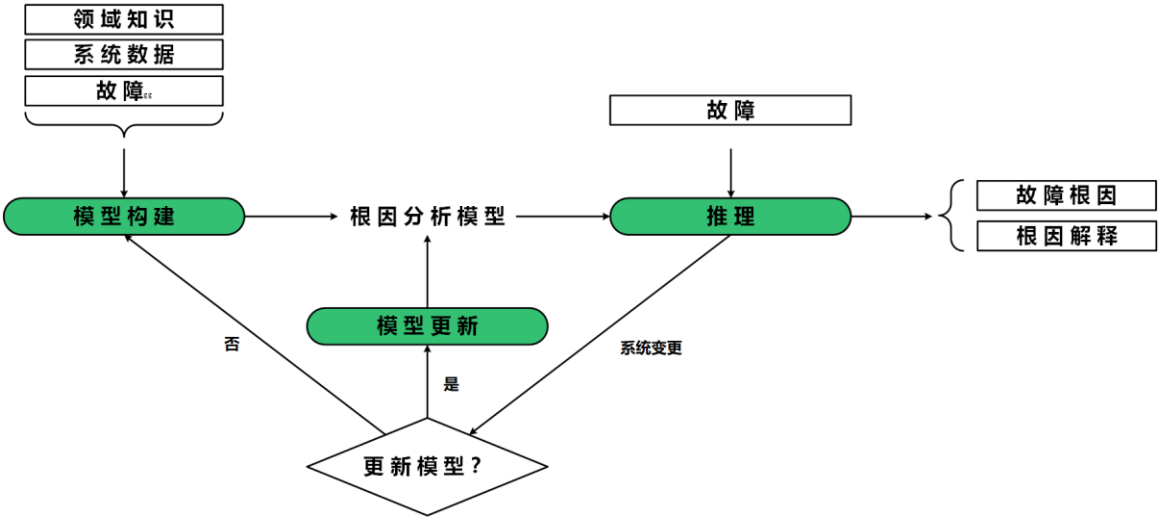

图 2-1 故障根因分析工作过程

所有根因分析技术[36]都可以根据不同的特点进行分类。一个相关分类是诊断模型是否知道被诊断系统的因果关系结构。它们有时被称为基于模型的方法[37]。比如 Wolfgan(2003)[38]提出了一个基于模型的调试框架，它能够通过使用一组通过分析源代码建立的模型来检测软件中的编程错误。与这些方法相比，本文不仅关注故障，还关注性能下降或其他类型的问题，这些问题可能不涉及严重的故障。另一方面，基于分类的方法通常使用机器学习技术生成一些内部模型，然后通过拥有大量较轻的知识类型(即分类标签)来交换对明确领域知识的需求。图已经以多种方式连接到根因分析。大多数以前的尝试都是基于使用概率图模型作为诊断模型或作为诊断系统的基本描述模型，一些其它用途仅限于异常检测和构建替代图模型。本文认为图是描述系统状态的实体，可以作为经验知识存储下来，构建故障知识图谱用于根因分析。

2.2 知识图谱

故障的传播路径蕴含着很多故障信息，传播路径上不同的设备和不同的传播顺序会导致不同的故障。知识图谱是结构化的语义知识库，可以用来保存故障传播路径这种结构有限制的有向无环图。将故障传播路径这种显式知识沉淀为故障知识图谱，然后引入故障根因分析不仅可以提高根因分析精确率，还可以帮助运维人员了解故障传播过程，从而针对性地修复故障。

早在 2006 年，Web 之父 Tim Berners Lee 提出链接数据的概念[26]，意图将网页的链接转为数据的链接。2012 年，知识图谱[25](Knowledge Graph, KG)被 Google 正式提出，旨在提升搜索质量。随后，知识图谱在学术界和工业界迅速普及，成为人工智能、数据库等领域的常用技术。

中国知网 https://www.cnki.net

知识图谱是由实体和关系组成的多关系图，实体在多关系图中被视为节点，关系在多关系图中被视为不同类型的边。从知识表示的角度看，知识图谱是一种采用图模型来对人类知识进行表示的知识库或者知识的集合，并且符合某种语法和语义。知识图谱的图模型是一个有向图$G =< O, E >$，这里O是一个实体或者字面量(literal)的集合，一般有以下几种：

1) 实例，即一个现实世界中具体的事物，比如说 1 号服务器；
2) 概念，可以看成是实例的集合，比如说服务器；
3) 字面量，主要分为字符串和数据类型，比如日期值、数量等。

实例和概念统称实体。E是带标签的有向边组成的集合，每条边e的标签指的是这条边指代的关系或者属性。知识图谱中一条知识称为三元组，即$< o_1, e, o_2 >$的格式，这里o_1和o_2是实体或者字面量，而 e 则是带标签的边。

上述知识图谱是经典知识图谱，本文为了构建可以存储故障传播路径的知识图谱，将事件和抽象事件引入知识图谱，从而构建故障知识图谱。事件类似于知识图谱的实例，抽象事件类似于知识图谱中的概念，事件和事件之间的关系类似于知识图谱中的关系e。事件、抽象事件和故障知识图谱的定义在 3.1 节中会详细介绍。

迄今为止，一系列的知识图谱已经被构建和应用[27]，如 DBpedia[28]、Freebase[29]、Yago[30]等。这些知识图谱内容更多的集中于英文语言，而在中文领域，也包括一些质量较高的知识图谱，比如 zhishi.me[31]、XLore[32]、CN-DBpedia[33]等。另外，Kiesling 等人提出了网络安全知识图 SEPSES[34]，该图集成并链接了来自各种公开可用来源的有关现实世界中的漏洞，弱点和攻击模式的关键信息。同时，Nie(2016)[7]针对网络服务故障诊断提出了一种基于因果图的自动诊断系统，以帮助系统操作员找到网络服务故障根本原因，保护系统安全。

知识图谱不仅可以作为一种知识的存储方式辅助根因分析，而且其图结构更利于找到故障传播路径，因此本文选取知识图谱作为故障根因分析核心。本文将从故障历史数据中构建故障知识图谱，用于 AIOps 领域的故障根因分析工作。

2.3 图卷积神经网络

本文使用图卷积神经网络抽取故障传播图(Fault Propagation Graph, FPG)和故障知识图谱(Fault Knowledge Graph, FKG)的特征，从而计算两个图的相似度，进行故障根因分析。

图卷积神经网络模型填补了目前神经网络无法处理非欧氏结构的空白，其利用图结构、节点特征和边特征学习图中节点的低维表征向量以及整个图的表征向量[41]。现有的图卷积神经网络模型大多是消息传递网络架构，这种架构主要包括两阶段：消息传递阶段与图整体表征向量读出阶段，其中消息传递阶段也常被称为聚合过程[42]。消息传递阶段中较为流行的方法是基于邻居的聚合方法，这种策略通过多次迭代聚合邻居节点与边的表征信息不断更新节点的表征向量。图卷积网络用层数表示迭代次数，随着迭代次数的增加，聚合的节点邻居信息就越多。k 次迭代能够聚合到距离当前节点 k 个跳步内的邻居信息，此时图卷积网络的层数为 k 层。由于第 k 层的信息是由第 k-1 层聚合而来，因此图卷积神经网络的第 k 层聚合可表示为公式(2-1)(2-2)。

中国知网 https://www.cnki.net

$$\mathrm{h}_v^{(k)} = COMBINE^k(AGGREGATE\left(h_u^{(k-1)}, \forall u \in N(v)\right), h_v^{(k-1)}) \qquad (2-1)$$

$$\mathrm{h}_G = READOUT(\{h_v^{(k)} | v \in V\}) \qquad (2-2)$$

其中，$h_v^{(k)}$表示第 k 层节点v的特征信息，$N(v)$表示节点v的邻居节点，h_G为全图的特征表示方法，V表示边的集合。

聚合阶段是图卷积神经网络的重要步骤，可根据聚合方法的不同划分形成不同类型的图卷积神经网络。图卷积网络可根据聚合方法分为谱方法与空间方法两类。第一个谱卷积神经网络是 Bruna(2014)[43]基于图谱理论提出的。随后 Defferrard(2016)[44]采用切比雪夫方法对卷积核进行参数化并提出切比雪夫网络。Kipf(2017)[45]对切比雪夫网络进行了简化并对权重矩阵做归一化处理，提出了 GCN。图 2-2 为 GCN 的基本框架。

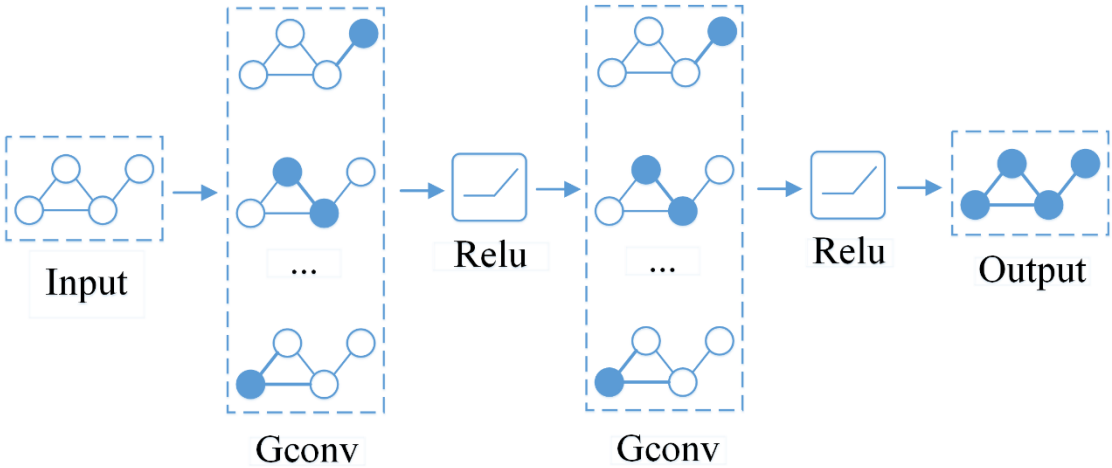

图 2-2 GCN 的基本框架图

谱卷积神经网络虽存在聚合的信息来源不一定是邻域节点的问题，但切比雪夫网络与 GCN 已经开始从空间域考虑并定义节点权重矩阵，为研究人员考虑在节点域定义聚合函数奠定了基础。为了解决 GCN 在大型图中计算拉普拉斯矩阵费时的问题，GraphSAGE[41]在节点邻域内随机采样并使用可训练的聚合函数替代拉普拉斯矩阵。FastGCN[46]进一步改善了采样算法，不在局限于节点邻域内的样本，而是直接对每一层的感受野随机采样。聚合函数的不同也形成了不同类型的图卷积神经网络。

GCN 没有考虑节点之间的关系，关系图卷积网络(Relational Graph Convolutional Network, R-GCN)[47]进一步解决了利用 GCN 来处理图结构中不同边关系对节点的影响。R-GCN 和 GCN 都是利用图卷积模型来模拟信息在网络结构中的传递，基于上述的方式，R-GCN 定义了多关系图的传播模型。图 2-3 展示了单个 R-GCN 层，从图中可以产出，模型对中心红色的节点进行一次卷积，通过聚合邻居节点的信息来更新自身节点的表示。其中邻居节点的聚合是按照边的类型进行分类，根据边类型的不同进行相应的转换，收集的信息经过一个正则化的加和

中国知网 https://www.cnki.net

(绿色方块)，最后通过激活函数(Relu)输出到下一层。其中每个顶点的信息更新共享参数，并行计算，同时也包括自连接，也就是说包括了节点自身表示。

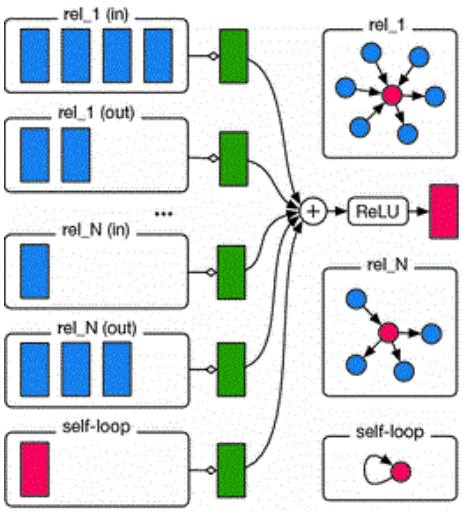

图 2-3 R-GCN 模型示意图

本文中图的关系为多类型，传统 GCN 无法使用，因此选取 R-GCN 作为本文相似度计算模块中提取图特征的核心模型。

2.4 聚类算法与图核

2.4.1 聚类算法

本文使用聚类算法对相似的故障传播图进行聚类，从而找出故障传播图的相似结构，自动构建故障知识图谱。

聚类分析是将数据集的相似性样本归为若干类的方法。目前聚类算法很多，可分为划分聚类和层级聚类两个大类。划分聚类算法针对一个包含 N 个样本的数据集，先创建一个初始划分，然后采用一种迭代的重定位技术，通过样本在类别间移动来改进聚类簇，最后通过一个聚类准则结束移动，并判定结果的好坏。其代表算法是 K-Means、混合密度聚类、模糊聚类等。本文中将图作为样本，对图进行聚类，由于图无法像坐标一样移动，所以无法使用划分聚类的方法。层次聚类算法通过对数据集按照某种方法进行层次分解，直到满足某种条件为止。层次聚类具有一个分层的树形结构。按照构建树形结构的方式不同，可以将聚类分为自顶向下和自底向上两种构建，分别称为聚合型层次聚类与分裂型层次聚类。两种聚类算法都是在聚类过程中构建具有一定亲属关系的系统树图。聚合型层次聚类，也称自底向上的方法，首先将每一个样本都称为一个聚类簇，然后计算簇间的相似度，分层合并，直到最后只有一个簇为止或满足一定的终止条件；而分裂型层次聚类的迭代过程则正好相反。分裂型层次聚类，也称自顶向下的方法，首先将所有的样本都看作是一个聚类簇，然后在每一步中，上层聚类簇被分裂为下层更小的聚类簇，直到每个簇只包含一个样本，或者满足终止条件为止。由于分裂型算法具有较高的时间复杂度，与聚合型算法相比，分裂型算法并不常见。常见的层次聚类改进算法有 BIRCH、ROCK 和 CURE 等。BIRCH 算法是一种改进的聚合型层

中国知网 https://www.cnki.net

次聚类算法。该算法的基本思想是每一个数据点的重要性肯定存在差异性，这样就能够差别化地对待不同的数据样本。算法分为两个步骤：一是扫描所有的样本点，构建内存树；二是利用算法建立树的叶子节点。Guha(1998)[48]提出了 CURE 算法，该算法的优点是使用具有代表性的一些点来代替聚类簇中的一个中心样本点，这样可以识别各种复杂形状和不同大小的簇，并且具有异常点检测功能。之后 Guha(1999)[49]提出的 ROCK 算法是对 CURE 算法的改进，使之具有识别类别属性的功能。

本文场景的聚类类别未知，且只知道样本(FPG)之间的距离，不知道样本的具体坐标。传统的基于聚类中心的算法如 k-means 由于没有样本坐标，无法找到聚类中心，所以无法使用。因此我们考虑层次聚类方法。而层次聚类中的分裂型算法具有较高的时间复杂度，所以本文采用聚合层次聚类(Hierarchical Agglomerative Clustering, HAC)。

2.4.2 图核

在对故障传播图进行聚类时，需要量化故障传播图间的相似度。图相似性度量有两种方案，一种是计算图之间的相似度，另一种是图匹配[39]。前一种可以使用图嵌入或者图核的方法。图嵌入将图的节点或图本身映射到向量空间中的点，而图内核将两个图映射为实数的函数。第二种图匹配方案的一些技术是基于树搜索的，例如 Hidovic[40]。由于需要量化图之间的相似度，而且需要考虑故障传播图的子结构，所以这里采用图核的方法。

图核(Graph Kernel)方法将机器学习中的核方法（Kernel Methods）拓展到了图结构数据上，是一类计算图与图之间相似度的方法。该方法可以高效地计算图之间的相似度，并可以方便地使用分类器进行图分类。图核的思路是将图映射到某个 Hilbert 空间，两个图之间的相似度可以通过 Hilbert 空间中的点积运算得到。常见的图核定义为图中子结构的分布，子结构包括随机游走，最短路径，环，子树等。Shervashidze(2011)[50]提出了 Weifsfeiler-Lehman (WL)框架提高图核的效果。WL 方法借鉴了标签传播的思想，需要进行 h 次迭代过程，每次迭代会构建一个新图，图的节点标签根据上一次迭代的邻居节点标签更新，并计算更新后的图核。WL 方法中，图的相似度由 h 次迭代过程中的图核综合得到。

由于故障传播图中某些子图的特性很重要(某些子图代表着故障的传播)，而图核主要就是利用局部的子结构信息，所以这里选取图核的方法计算两个图之间的相似度，且故障传播图中子树结构比较重要，所以这里选取 Subtree Kernel 作为核函数。

图上的核由节点上的核累加得到，节点的核计算方式如公式(2-3)。

$$k_h(v,v') = \begin{cases} \delta\big(\ell(v),\ell(v')\big), if \;\; h=0 \\ \delta\big(\ell(v),\ell(v')\big) \sum\limits_{R\in\mathcal{M}(v,v')} \prod\limits_{(w,w')\in R} k_{h-1}(w,w'), if \;\; h>0 \end{cases} \qquad (2-3)$$

其中$k_h(v,v')$计算的是子树高度为 h 时节点v和v'的 Subtree Kernel，子树高度 h 越大，Subtree Kernel 估计的越准确。δ是一个指示函数，当它的参数相等时为 1，否则为 0。ℓ表示节点的标签。其中$\mathcal{M}(v,v')$的计算方式如公式(2-4)。

$$\mathcal{M}(v,v') = \{R \subseteq \mathcal{N}(\mathrm{v}) \times \mathcal{N}(v') | (\forall (u,u'),(w,w') \in R: \mathrm{u} = \mathrm{w} \Longleftrightarrow u' = w') \\ \wedge (\forall (u,u') \in \mathrm{R} : \ell(u) = \ell(u'))\} \qquad (2-4)$$

中国知网 https://www.cnki.net

$\mathcal{N}$(v)是节点v的邻接点的集合，$\mathcal{M}(v,v')$是v和v'的邻域的子集的精确匹配集合。$\mathcal{M}(v,v')$的每个元素 R 是来自$v \in V$和$v' \in V'$的邻域的一组节点对，每个节点对中的节点具有相同的标签，并且每个节点只在一个节点对。

2.5 本章小结

本章首先介绍了 AIOps，然后介绍了 AIOps 故障根因分析的任务、难点、工作流程和一些相关框架。接着为了方便在第三章介绍故障知识图谱的构建技术，又对知识图谱进行了介绍。最后介绍了用于编码的深度学习模型和其他算法，包括图卷积神经网络、聚类算法和图核，这些技术将会在本文方法中被具体应用。

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net

第三章 故障知识图谱构建

本章介绍故障知识图谱的构建方法，构建的故障知识图谱将用于第四章的故障根因分析。本章首先介绍一些专业名词定义，然后介绍事件的生成方法，接着介绍故障传播图谱构建算法和故障知识图谱构建算法，之后介绍故障知识图谱构建方法评估实验，最后对本章的内容进行小结。

3.1 相关定义

定义 1：**运维事件**。运维事件$e=(E,t,l)$，其中$E\in\varepsilon$是事件类型，$\varepsilon=\{E_1,E_2,\dots,E_m\}$是$m$个事件类型的有限集合，$t$表示该运维事件的时间戳，$l$表示此事件发生的位置[51]。

本文中的事件都是运维事件，为了叙述方便，下文中使用事件表示运维事件，所有事件都是指运维事件。一个事件[7]包含时间、地点、事件类型。时间是指事件在什么时候发生的。地点是指事件发生的位置，一般是组件名称，同时也是事件的主要参与者。比如 1 号服务器在 12 点内存满载，这里的地点就是 1 号服务器，表明此事件是发生在 1 号服务器上，1 号服务器是此事件的主要参与者，这里的地点类型是服务器。事件类型是指具有相同特点的一类事件的统称。下面就是一个具体事件的实例。

{

时间：1576078184(时间戳)

地点：Container-6b6fc4869d

事件类型：Container Memory 100%

}

本文的事件类型主要分为正常事件、异常事件和故障事件。正常事件是指系统内组件进行一些成功操作产生的事件，注意正常事件并不是不能引发故障，比如服务器重启事件，虽然此事件是服务器正常重启产生的事件，但是如果在此服务器重启时，其他服务器对其访问，则会产生访问失败事件，可能会引发故障。异常事件是指系统组件进行一些失败的操作产生的事件，或者运维人员使用异常检测算法检测到的系统异于常态的事件，但此事件发生时组件仍可正常运行。故障事件是指描述组件故障的事件，故障事件发生时组件已无法正常运行。再往下可以细分为硬件故障和软件故障。具体运维事件类型划分及其实例如表 3-1。

定义 2：**抽象运维事件**。抽象运维事件$e'=(E,L)$，其中$E\in\varepsilon$是事件类型，$\varepsilon=\{E_1,E_2,\dots,E_m\}$是$m$个事件类型的有限集合，而$L\in\gamma$是地点类型，$\gamma=\{L_1,L_2,\dots,L_n\}$是$n$个地点类型的有限集合。

本文中的抽象事件都是抽象运维事件，为了叙述方便，下文中使用抽象事件表示抽象运维事件，所有抽象事件都是指抽象运维事件。抽象事件是相对具体事件而言的，抽象事件没有具体时间和具体地点，用来描述发生在某地点类型的某种事件类型。地点类型L是地点的类型，如服务器、Pod、微服务、容器等。下面是某抽象事件：

中国知网 https://www.cnki.net

{

地点类型：Container

事件类型：Container Memory 100%

}

定义 3：**故障传播图(Fault Propagation Graph, FPG)**。故障传播图 FPG 是三元组(e_1, r, e_2)的集合，其中e_1和e_2是事件，r是事件e_1和e_2之间的关系。r是顺序关系或者因果关系。

表 3-1 运维事件分类表

粗类别	细类别	实例
正常事件	硬件正常事件	container memory 50%、container cpu 50%、...
	软件正常事件	数据库连接成功、...
异常事件	硬件异常事件	container memory 100%、container cpu 100%、...
	软件异常事件	数据库连接失败、...
故障事件	硬件故障事件	服务器宕机、...
	软件故障事件	HttpServerErrorException 503、HttpServerErrorException 500、...

故障传播图记录着具体事件的传播过程，其中顺序关系是某事件 A 必须发生在事件 B 之前，但并不构成因果关系。图 3-1 是部分故障传播图示例。

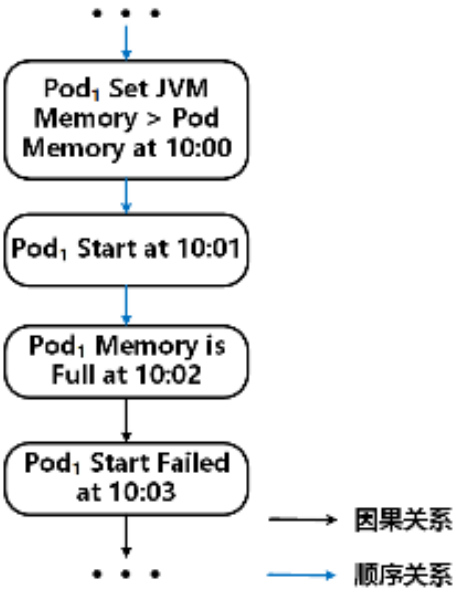

图 3-1 部分故障传播图示例

定义 4：**故障知识图谱(Fault Knowledge Graph, FKG)**。故障知识图谱 FKG 是三元组(e_1', r, e_2')的集合，其中e_1'和e_2'是抽象事件，r是抽象事件e_1'和e_2'之间的关系。r是顺序关系或者

中国知网 https://www.cnki.net

因果关系。

故障知识图谱是描述某种故障常见触发过程的结构化图谱，一张故障知识图谱代表一种故障。多个故障知识图谱构成故障知识库，代表着故障的经验信息。故障传播图记录着具体事件的传播过程，而故障知识图谱记录着某类故障的传播过程，并作为经验保存下来，是故障传播图的抽象化。

定义 5：**根因分析(Root Cause Analysis, RCA)**。给定系统中的故障事件和其发生前一段时间的事件序列，根因分析的任务是识别导致故障事件的根本原因事件。

3.2 事件生成

故障知识图谱记录着某类故障的传播过程，是知识的显式存储，将显式知识融入故障溯源，可以在一定程度上提升溯源精度。故障知识图谱是由抽象事件及其之间的关系组成，为了构建故障指示图谱，首先需要生成分布式系统中的事件。

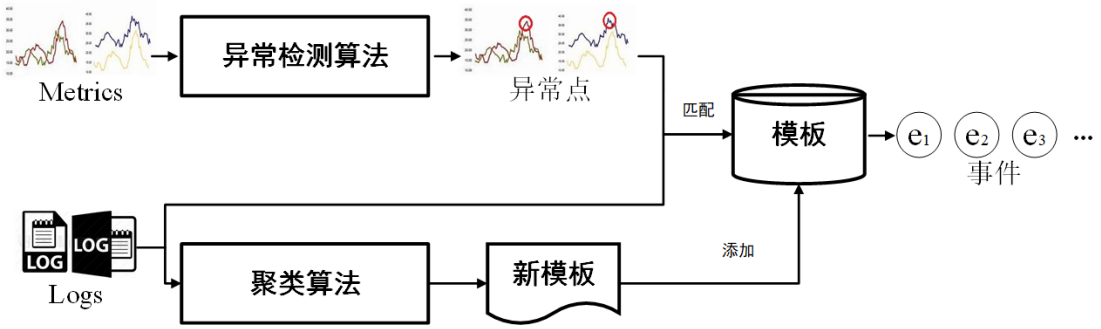

图 3-2 事件生成方法示意图

如图 3-2 所示，本节的主要功能是生成系统中的事件。系统中的数据主要有两种，一种是 Metric，记录着系统中每个组件的指标信息，另一种是 Log，记录着组件的日志信息。

对于日志数据，一方面本文使用聚类算法[52]在日志中发现通用模板，然后将新发现的模板添加到模板库中。另一方面，本文会使用模板库中的模板匹配日志生成相应事件。

对于 Metric 数据，本文会使用异常检测算法[51]检测到异常点，比如某些指标开始很平滑，然后经过某个点后突然抖动，那么这个转折点就是异常点，异常点一般是由于系统中的异常产生的。找到异常点后，经过模板匹配可以得到异常事件。假设上面的例子中的指标是 CPU 使用率，如果开始平滑，然后突然抖动突增，那么这个点是异常点，根据模板可以得出这个异常点事件类型是 CPU 使用率突增。

3.3 故障传播图构建

故障传播图中包含着事件和事件之间的关系，为了构建故障传播图，需要发现事件间的关系。故障传播图描述着故障以事件为表示形式的传播路径，这种传播只需要顺序关系和因果关系就能表示出来。但是不能仅仅是因果关系，因为有时虽然两个事件是顺序关系，但它传播了故障。图 3-1 是部分故障传播图，虽然开启 Pod 并不一定就会导致 Pod 满载，他们是顺序关系不是因果关系，但是确实也传播了故障，是描述故障传播不可缺少的一部分。

中国知网 https://www.cnki.net

经过上面的分析，两个事件之间可能是顺序关系、因果关系和无关系。顺序关系和因果关系统称为相关关系。一个事件和其他事件频繁共现，那么它们是非常相关的，本文称为相关关系。本文会使用相关性粗粒度地判断两个事件是否具有相关关系，然后再使用关系分类器φ把相关关系细分为顺序关系和因果关系。

这里不直接使用关系分类器φ判断两个事件之间的关系的原因有两个：

1) 降低计算量。直接使用关系分类器需要两两判断事件之间的关系，如果可以先将关系进行粗粒度分类，然后再用关系分类器判断，那么将大大降低计算量。
2) 引入事件所处的环境信息辅助关系发现。只考虑两个事件本身来判断他们之间的关系是不合理的。有时事件 A 和事件 B 之间没有相关关系，但是事件 A 在某些环境下会导致事件 B。

算法 1：故障传播图构建算法

输入：故障发生时一个时间窗口内获得的事件序列 Q, 最大相关事件数 M，最小相关性阈值 θ。

输出：故障传播图 FPG

```
FPG ← {}
Q ← {e_1, e_2, ..., e_N}
while Q ≠ ∅ do
    e_0 ← PopEarliestEvent(Q)
    if FPG == ∅ then
        FPG ← FPG + {e_0}
    else
        CandidateGraphs ← GetCandidateGraphs(FPG, M, e_0)
        BestGraph ← GetBestGraph(CandidateGraphs, θ)
        if BestGraph == ∅ then
            FPG ← FPG + {e_0}
        else
            for (e_i, r, e_j) ∈ (BestGraph − FPG) do
                FPG ← FPG + {(e_i, φ(e_i, e_j), e_j)}
            endfor
        endif
    endif
endwhile
```


中国知网 https://www.cnki.net

例如，“设置 POD 内存 10M”不能直接导致“POD 启动失败”，“设置 POD 内 JVM 内存 2G”也不能直接导致“POD 启动失败”，但是“设置 POD 内存 10M”之后“设置 POD 内 JVM 内存 2G”就会因为 JVM 内存小于 POD 内存导致“POD 启动失败”，所以判断两个事件之间的关系，不仅仅要考虑两个事件本身，还需要考虑事件所处的一个环境，也就是事件已有的相关关系。也就是根据之前构建的故障传播图，迭代地判断新的关系。

下面介绍故障传播图构建算法，算法 1 为故障传播图构建算法。算法输入是故障前一段时间系统中的事件序列、最大相关事件数 M 和最小相关性阈值θ，输出是故障传播图谱。最大相关事件数 M 是指假设一个事件最多由 M 个事件导致，根据 3.5.3 节的实验结果来看，M 最优值为 3，也就是说一个事件最多由三个事件导致。θ是最小相关性阈值，当事件和所有可能连接的事件集合的相关性小于θ时，该事件会作为单独节点加入到图中。

开始故障传播图为空，之后每一步迭代都将选取一个事件加入故障传播图，直到事件序列为空，然后返回故障传播图。由于事件是有严格的发生顺序的，因此很自然地想到按照时间先后构建故障传播图，这样后添加的事件在判断相关性的时候可以利用到之前发生的事件及其相关性。所以算法在每次迭代时都先使用 PopEarliestEvent 函数找出事件序列 Q 中早发生的事件e_0此时故障传播图是空的，那么直接将事件e_0加入到故障传播图中，进行下一次迭代，否则使用 GetCandidateGraphs 函数找出将事件e_0加入到故障传播图 FPG 中可能形成的所有候选图 CandidateGraphs，然后使用 GetBestGraph 函数找到得分最高的候选图，并用 RelationClassify 函数将得分最高的候选图中刚生成的关系进行关系分类，然后作为本次迭代生成的故障传播图 FPG。依次迭代直到事件序列 Q 为空，然后返回故障传播图。

GetCandidateGraphs 函数会找出将事件e_0加入到故障传播图 FPG 中可能形成的所有候选图 CandidateGraphs。在向故障传播图中添加新事件的时候，由于相关性具有传递性，所以无论事件和故障传播图中的哪个事件相关，最后由于传递性都可以转化为与叶节点相关，所以每次迭代算法会将新事件e_0连接到叶子节点上。这样新事件e_0可以和任意一个叶子节点相连，也可以和任意两个叶子节点相连，但是由于算法设置了最大相关事件数 M，新事件e_0最多和 M 个事件有直接相关关系，所以新事件e_0至多和 M 个叶子节点相连，每次将新事件e_0连接到叶子节点上就形成一张候选图，最终会获得$\sum_{i=1}^{M} C_N^i \leq 2^N$张候选图，其中 N 是当前叶子节点的数目。GetCandidateGraphs 函数会返回这些候选图集合 CandidateGraphs。

GetBestGraph 函数给每张候选图打分，然后找到得分最高的候选图作为本次迭代生成的故障传播图 FPG 返回。这里算法使用相关性作为打分原则。计算相关性的时候不是简单考虑两个事件之间的相关性，而是考虑新加入事件和事件集合之间的相关性，这个事件集合代表着一种环境。下面计算e_0和叶子节点及其相连事件集合的相关性。假设第 i 个候选图中与e_0直接相连的叶节点集合为l_i，算法找出所有能通过有向边到达l_i中每个叶子节点的事件集合，并加入l_i中所有叶子节点，生成集合$Set_i \leftarrow \{e_{i1}, e_{i2}, \dots, e_{iN},\}$，$Set_i$就代表着$l_i$中叶子节点所处的环境。计算候选图 i 的得分，也就是计算e_0和Set_i的相关性，如果相关性越高，说明e_0就应该连接到Set_i之后，连接时自然而然连接到叶子节点上。如果所有候选图的得分都小于最小相关

中国知网 https://www.cnki.net

性阈值θ，那么 GetBestGraph 函数返回空集，否则返回得分最高的候选图。候选图 i 的得分$score_i$计算公式如公式(3-1)。

$$score_i = \mathrm{P}(e_0|Set_i) = \frac{Count_1(\{e_{i1}, e_{i2}, \dots, e_{iN}\})}{Count_2(e_0, \{e_{i1}, e_{i2}, \dots, e_{iN},\})} \quad (3-1)$$

其中$Count_1$、$Count_2$是统计函数。本文将数据集划分成为两部分，一部分作为训练集，一部分作为测试集。然后又把训练集分为两部分，一部分用于统计概率，一部分用于构建FKG。对于统计概率的这部分数据，本文将其分成若干时间段（10 分钟为 1 段），$Count_1$统计$e_{i1}, e_{i2}, \dots, e_{iN}$同时出现在同一时间段的次数，$Count_2$统计$e_0, e_{i1}, e_{i2}, \dots, e_{iN}$同时出现在同一时间段且$e_0$出现在$e_{i1}, e_{i2}, \dots, e_{iN}$之后的次数。

得分最高的候选图中e_0和其他叶子节点目前只是相关，算法接下来使用关系分类器φ确定e_0和其他叶子节点之间的关系。$\varphi(e_i, e_j)$可以判断事件e_i和事件e_j之间的具体关系，返回因果关系或者顺序关系。下面介绍关系分类器。

本文利用了事件的特征在历史数据上训练了得到关系分类器，可以判断两个事件之间的关系是因果关系还是顺序关系。关系分类器模型可以是常见的机器学习分类器模型，本文分别使用了 SVM、Xgboost、随机森林、逻辑回归等经典机器学习模型作为分类器的分类模型，并且实验表明 SVM 关系分类效果最好。模型使用以下特征：

（1）事件周期性

对于周期性出现的事件，很大可能是白噪声。这种周期性出现的事件不论故障发生与否都会周期性出现。白噪声一般不会引发故障事件，所以事件周期性越高，它与其他节点的因果关系越弱。

（2）事件关键性

事件的关键性是指事件对某一故障的重要程度，关键性越高，说明这个事件可能和故障有着某种潜在的关系。一般来说，和某一故障有潜在关联的事件会伴随着故障的发生而发生。两个事件的关键性越高，关键性相差越小，越有可能是因果关系。

（3）皮尔逊相关系数

皮尔逊相关系数是用来计算两个事件之间的相关性，两个事件之间的皮尔逊相关系数越高，那么这两个事件越相关，越有可能是因果关系。

（4）事件共现频率

A、B 事件在一起出现的频率越高，越有可能有因果关系。

（5）事件发生位置间的距离

事件发生的位置越近，越可能是因果关系。

（6）事件发生时间

事件发生时间越近，越可能是因果关系。计算事件 A 和事件 B 时间差时，事件 B 可能发生多次，这时计算事件 A 与 B 的平均时间差。

中国知网 https://www.cnki.net

3.4 故障知识图谱构建方法

故障知识图谱是故障传播图的抽象化，是相似故障传播图的共有结构，所以本文通过找出相似故障图的共有结构获得故障知识图谱。图 3-3 是故障知识图谱的构建示意图，在故障知识图谱构建过程，本文会先使用故障传播图构建方法构建出这 N 次故障的故障传播图，组合器会忽略故障传播图中事件节点具体的时间和地点，找出所有故障传播图中共有的结构，进行故障知识的沉淀，将共有的结构抽象化生成故障 A 的故障知识图谱。得到故障知识图谱后需要人工标注每张故障知识图谱的根因事件，由于直接进行人工标注比较耗时，本文设计了辅助根因标注(Auxiliary root cause labeling, ARCL)算法辅助专家进行根因标注。

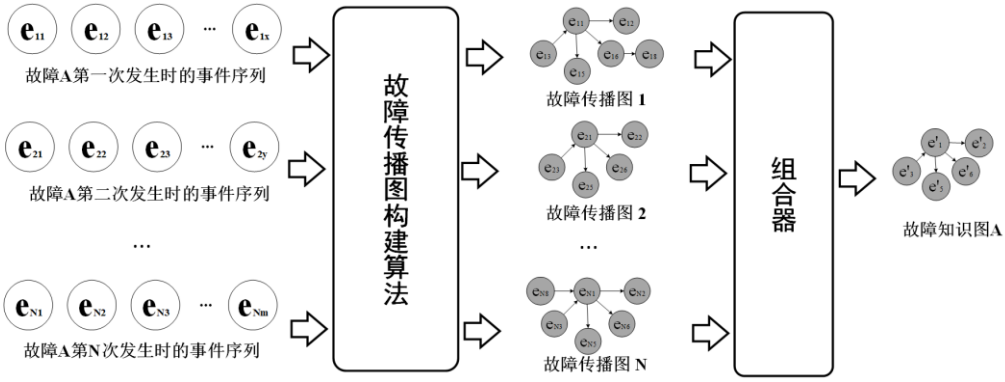

图 3-3 故障知识图构建示意图

3.4.1 故障知识沉淀

f 故障可能由不同根因导致，所以其故障传播图不全是类似的，所以最终生成的 f 故障的故障知识图是由多个子图构成的，每个子图是由相似的 f 故障的故障传播图抽取共有结构得到的。所以这里的算法应该有两个步骤，第一步是找出相似的故障传播图，第二步是抽取相似传播图共有的结构。下面的算法 2 是组合器中的故障知识图构建算法。

算法 2：故障知识图谱构建算法

输入： f 故障不同时间段的故障传播图集合$S = \{FPG_1, FPG_2, \dots, FPG_n\}$，簇间 linkage 阈值$\mu$

输出： 故障知识图谱$FKG = \{SG_1, SG_2, \dots, SG_m\}$

1. $FKG \leftarrow \{\}$
2. $\pi \leftarrow GraphClustering(S, \mu)$
3. $\boldsymbol{for}\ \ eachcluster\ \ \pi_i \in \pi\ \ \boldsymbol{do}$
4. $\quad SG \leftarrow \{\}$
5. $\quad \boldsymbol{for}\ \ each\ \ FPG \in \pi_i\ \ \boldsymbol{do}$
6. $\quad\quad AbstractFPG \leftarrow Abstract(FPG)$
7. $\quad\quad SG \leftarrow SG \cap AbstractFPG$
8. $\quad \boldsymbol{endfor}$
9. $\quad FKG \leftarrow FKG + SG$
10. $\boldsymbol{endfor}$

中国知网 https://www.cnki.net

该算法分为两个步骤，第一步使用图聚类函数 GraphClustering 对故障传播图集合 S 进行聚类，把相似的故障传播图聚成一类，第二步将同一类的故障传播图进行合并，生成 f 故障的故障知识图子图，从而组成整个 f 故障的故障传播图。

算法 3 是 GraphClustering 算法，算法采用层次凝聚聚类(Hierarchical Agglomerative Clustering, HAC)算法对故障传播图聚类，由于 Single-link 和 Complete-link 对离群点或噪声数据过分敏感,而 Average-link 往往被认为是最常用也最好用的方法，一方面它可以克服离群点敏感性问题，另一方面因为其空间扩张/浓缩的程度适中，所以这里的 Linkage 采用 Average-link。当两个簇之间 Linkage 小于阈值μ或者聚类类别为 1 时，算法终止。这里采用图核(Graph Kernel)计算两个图之间的相似度。由于故障传播图中某些子图的特性很重要(某些子图代表着故障的传播)，而图核主要就是利用局部的子结构信息，所以这里选取图核的方法计算两个图之间的相似度，且故障传播图中子树结构比较重要，所以这里选取 Subtree Kernel 作为核函数。

合并是抽取上一步聚类得到的同类故障传播图的共有结构，所以这里使用求交集的方法获取图的共有结构。这种共有结构是一种抽象的经验信息，所以在合并之前需要使用 Abstract 函数将故障传播图抽象化，将具体的时间和地点信息去除，得到 AbstractFPG。之后对同类 AbstractFPG 取交集，得到公共抽象事件的关系三元组集合，此集合其实就是故障知识图，而且是 f 故障的故障知识图的一个子图。

算法 3：GraphClustering

输入： n 个故障传播图集合$S = \{FPG_1, FPG_2, \dots, FPG_n\}$，簇间 linkage 阈值$\mu$

输出： 聚类后的故障传播图集合π

1. $\pi_0 \leftarrow \{FPG_1, FPG_2, \dots, FPG_n\}$
2. $k \leftarrow 0$
3. $\boldsymbol{while}\ the\ number\ of\ clusters\ in\ \pi_k\ not\ equal\ to\ 1\ \boldsymbol{do}$
4. $\quad k \leftarrow k + 1$
5. $\quad L(P, Q) \leftarrow Calculate\ the\ linkage\ between\ 2\ clusters\ P\ and\ Q$
   $\quad which\ have\ the\ largest\ linkage\ among\ all\ pairs\ of\ clusters\ in\ \pi_{k-1}$
6. $\quad \boldsymbol{if}\ L(P, Q < \mu)\ \boldsymbol{then}$
7. $\quad\quad \boldsymbol{break}$
7. $\quad \boldsymbol{else}$
8. $\quad\quad R \leftarrow merge\ the\ 2\ clusters\ P\ and\ Q$
9. $\quad\quad \pi_k \leftarrow \pi_{k-1}\ after\ replacing\ \{P, Q\}\ by\ R$
10. $\quad\quad \pi \leftarrow \pi_k$
11. $\quad \boldsymbol{endif}$
12. $\boldsymbol{endwhile}$

中国知网 https://www.cnki.net

Linkage 的计算公式 L(P,Q)如公式(3-2)。

$$\mathrm{L(P,Q)} = \frac{1}{|\mathrm{P}||\mathrm{Q}|}\sum_{\mathrm{FPG}\in P}\sum_{\mathrm{FPG}'\in Q} s(\mathrm{FPG},\mathrm{FPG}') \tag{3-2}$$

Linkage 采用 Average-link，两两计算簇 P 中的故障传播图 FPG 和簇 Q 中的图FPG'的相似度$s(\mathrm{FPG},\mathrm{FPG}')$，累加后求平均。计算两个图的相似度$s(\mathrm{FPG},\mathrm{FPG}')$的公式如(3-3)所示。

$$s(\mathrm{FPG},\mathrm{FPG}') = \frac{\sum_{v\in V}\sum_{v'\in V'} k_h(v,v')}{C_{FPG,\mathrm{FPG}'}} \tag{3-3}$$

公式(3-3)中计算两个图的相似度采用的是 Subtree Kernel。其中V是 FPG 中的节点的集合，V'是FPG'中的节点的集合，v是 FPG 中的节点，v'是FPG'中的节点，$k_h(v,v')$计算的是子树高度为 h 时节点v和v'的 Subtree Kernel，子树高度 h 越大，Subtree Kernel 估计的越准确。$C_{FPG,\mathrm{FPG}'}$是归一化常数。图上的 Subtree Kernel 等于节点上的 Subtree Kernel 之和。节点的 Subtree Kernel $k_h(v,v')$计算方法如公式(1-3)。δ是一个指示函数，当它的参数相等时为 1，否则为 0。ℓ表示节点的标签。其中$\mathcal{M}(v,v')$的计算方式如公式(1-4)。$\mathcal{N}(\mathrm{v})$是节点v的邻接点的集合，$\mathcal{M}(v,v')$是v和v'的邻域的子集的精确匹配集合。$\mathcal{M}(v,v')$的每个元素 R 是来自$v\in V$和$v'\in V'$的邻域的一组节点对，每个节点对中的节点具有相同的标签，并且每个节点只在一个节点对。

3.4.2 故障知识标注

得到故障知识图谱后需要人工标注每张故障知识图的根因事件，由于直接进行人工标注比较耗时，本文设计了辅助根因标注算法辅助专家进行根因标注。算法 4 为辅助根因标注算法。

该算法输入是故障知识图 FKG，输出是根因抽象事件集合 R。算法首先使用 GetLeafNodes 找出所有叶子节点 LeafNodes，然后从每个叶子节点开始，由下向上寻找可能的根因。因为是由下向上搜索，所以这里使用 ReverseRelation 函数将图中关系反转，方便接下来的深度优先遍历。然后从每个叶子节点处开始深度优先遍历，DFT 就是深度优先遍历函数，每次遍历一个节点。每遍历到一个节点，都要判断这个节点可不可能是根因节点。对于每个节点，如果该节点是叶子节点（因为反转过关系，所以叶节点原来是根节点），或者该节点和前一个节点是因果关系并且和后边某个节点是顺序关系，那么它可能是根因节点，需要对他进行人工标注，并将标记结果记录到根因抽象事件集合 R 中。最后算法在输出 R 之前将图中关系反转回来。

其中 isLeaf 判断该节点是否是叶子节点，Pre 函数返回指向该节点的关系，Next 函数返回该节点指出的关系集合，Label 函数是标注函数，由专家将节点标记为根因或者非根因，如果是根因就返回该节点，否则返回空。

中国知网 https://www.cnki.net

算法 4：辅助根因标注算法

输入：故障知识图谱FKG

输出：根因抽象事件集合 R

1. $R \leftarrow \{\}$
2. $LeafNodes \leftarrow GetLeafNodes(FKG)$
3. $FKG \leftarrow ReverseRelation(FKG)$
4. $\boldsymbol{for}\ \ LeafNode \in LeafNodes\ \ \boldsymbol{do}$
5. $\quad \boldsymbol{for}\ \ Node \in DFT(LeafNode)\ \ \boldsymbol{do}$
6. $\quad\quad \boldsymbol{if}\ \ isLeaf(Node)\ \ or\ \ (Pre(Node) == casual\ \ relation\ \ and\ \ sequential\ \ relation\ \ \in Next(Node))\ \ \boldsymbol{then}$
7. $\quad\quad\quad R \leftarrow R + \{Label(Node)\}$
8. $\quad\quad \boldsymbol{endif}$
9. $\quad \boldsymbol{endfor}$
10. $\boldsymbol{endfor}$
11. $FKG \leftarrow ReverseRelation(FKG)$

3.5 实验

3.5.1 数据集

本文实验使用了两个数据集，数据集是从阿里云内部测试分布式系统的火车票系统(25 天)和电商系统(14 天)中采集而来。实验中的分布式系统为阿里云 Kubernetes 集群，分布式应用为 Train-ticket 和 Sock-shop。数据集采集时使用 Kube-monkey、Chaosblade 进行了故障模拟，采用阿里云的日志服务平台进行实时的 Log、Metric 信息收集，最终在 39 天内共制造 970 余次网络应用故障，收集了集群的设备信息、拓扑结构、系统日志数据和各个设备的监控信息（CPU 使用率、内存使用率等等）共计 10027998149 条。详细数据集信息见表 3-2。

表 3-2 数据集详细信息表

信息项	数据集 1（train-ticket）	数据集 2(sock-shop)
时间跨度	25 天	14 天
平均故障时间段跨度	303s	310s
单位故障含有事件数目	4823	3352
故障次数	423	550
故障种类	10	17
事件类型种类	204	211

为了训练关系分类器，本文对部分数据进行了标注，标注数据统计信息如表 3-3。

中国知网 https://www.cnki.net

表 3-3 因果关系标注

数据集	正例	负例	总计
Train-ticket	121	3610827	3610948
Sock-shop	165	4639463	4639463

3.5.2 评测指标

一般用的评测指标有 Accuracy、Precision、Recall 和 F1。Accuracy 是指模型判断正确的数据占总的测试数据的比例。Accuracy 对衡量类别不均衡的数据时无法很好的反映系统的性能，因为某一类别所占比例特别高时，一般 Accuracy 的值都会比较高。而本文的未标注数据中正例子和负例的比例不均衡，所以不适合使用 Accuracy。因此本文使用 Precision 和 Recall 以及 F1 来测试模型的性能。Precision 是指模型判断为正例的数据中真正属于正例的数据的比例，反映了分类器在判断正例时是否准确。Recall 是指真实的全部正例数据中模型判断为正例数据的比例， 反映了模型能够将全部正例数据检索到的能力。F1 得分同时兼顾了 Precision 和 Recall。

本文实验主要对故障知识图谱构建方法做评测，评测公式如公式(3-3)(3-4)(3-5)。

$$Precision = \frac{|R_{correct}|}{|R|} \tag{3-3}$$

$$Recall = \frac{|R_{correct}|}{|R_{label}|} \tag{3-4}$$

$$F1 = \frac{2 * Precision * Recall}{Precision + Recall} \tag{3-5}$$

其中R表示故障知识图谱中关系三元组集合，$|R|$表示故障知识图谱关系三元组数目，$R_{correct}$表示故障知识图谱中正确的关系三元组集合，$|R_{correct}|$表示故障知识图谱中正确的关系三元组数目。R_{label}表示标注故障知识图谱中关系三元组集合，$|R_{label}|$表示标注故障知识图谱中关系三元组数目。

3.5.3 实验结果

本章构建故障知识图谱时需要使用到关系分类器，关系分类器模型可以为常用的机器学习模型。本文测试了支持向量机、Xgboost、随机森林、逻辑回归和多层感知机五个常用的机器学习模型，并采用 5 折交叉验证方法。最终五个模型各自最好的预测结果如表 3-4。由于正负样本差距过大，而支持向量机分类决策中起决定作用的是支持向量，对于正负样本差距过大问题不敏感，因此这里效果较好。而感知机虽然召回率比较高，但是精确率比较低，综合来看本文使用支持向量机作为分类器模型。

表 3-5 是故障知识图谱构建结果，从结果中可以看到，FKG 平均构建结果的 F1 值可以达到 74%，基本满足需求。虽然有少数故障知识图谱构建效果较差，但是可以通过后续标注纠正。

中国知网 https://www.cnki.net

表 3-4 关系分类器评估表

数据集	模型	精确率	召回率	F1
Train-ticket	支持向量机	0.951	0.961	**0.956**
	Xgboost	0.911	0.895	0.903
	随机森林	0.950	0.792	0.864
	逻辑回归	0.913	0.845	0.878
	三层感知机	0.892	1.0	0.943
Sock-shop	支持向量机	0.958	0.969	**0.963**
	Xgboost	0.923	0.908	0.915
	随机森林	0.955	0.838	0.892
	逻辑回归	0.925	0.881	0.902
	三层感知机	0.903	0.981	0.940

表 3-5 故障知识图谱构建结果

数据集	FKG 名称	Presicion	Recall	F1
Train-ticket	Fault_1	0.745	0.816	0.779
	Fault_2	0.783	0.859	0.819
	Fault_3	0.832	0.899	0.864
	Fault_4	0.732	0.801	0.765
	Fault_5	0.709	0.763	0.735
	Fault_6	0.717	0.814	0.762
	Fault_7	0.748	0.808	0.777
	Fault_8	0.714	0.801	0.755
	Fault_9	0.683	0.818	0.744
	Fault_10	0.711	0.803	0.754
	Average	0.737	0.820	**0.779**
Sock-shop	Fault_1	0.708	0.789	0.746
	Fault_2	0.844	0.852	0.848
	Fault_3	0.721	0.799	0.758
	Fault_4	0.706	0.782	0.742
	Fault_5	0.625	0.762	0.687
	Fault_6	0.684	0.795	0.735
	Fault_7	0.683	0.792	0.733
	Fault_8	0.828	0.856	0.842
	Fault_9	0.694	0.799	0.743
	Fault_10	0.714	0.793	0.751
	Fault_11	0.707	0.79	0.746
	Fault_12	0.621	0.766	0.686
	Fault_13	0.701	0.79	0.743
	Fault_14	0.714	0.797	0.753
	Fault_15	0.831	0.854	0.842
	Fault_16	0.700	0.786	0.741
	Average	0.708	0.789	**0.746**

中国知网 https://www.cnki.net

接下来，我们考虑算法参数对构图的影响，考虑三个参数 Subtree Kernel 中的 h，最大相关事件数 M 和最小相关阈值θ。从图 3-6 可以看到，h 越大，聚类效果越好，聚类效果越好，但是当 h 超过 3 时，聚类精确率提升率不大，考虑到更大的 h 会增大时间开销，因此这里 h 选择 3。从图 3-5 可以看到最大事件数 M 为 3 时，构图效果最好。从图 3-4 可以看到最小相关阈值θ为 0.8 时，构图效果最好。

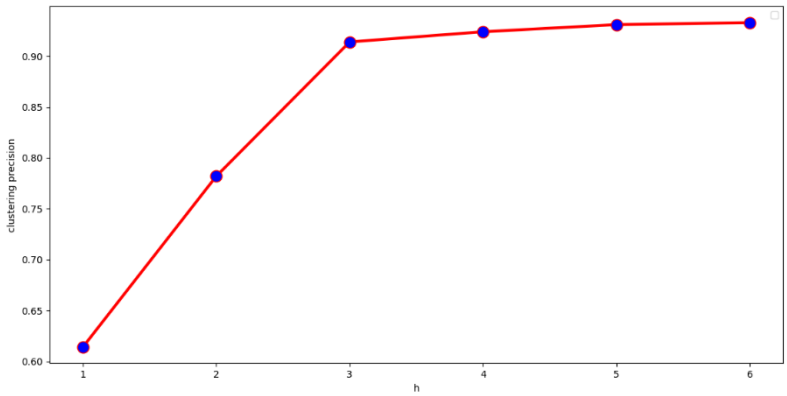

图 3-6 h-聚类精确率折线图

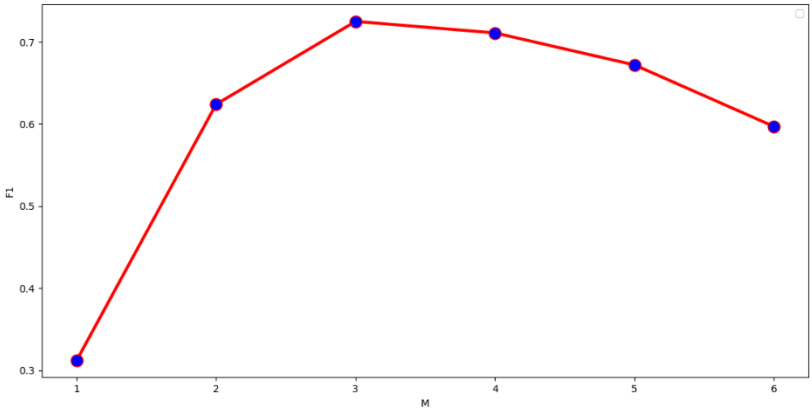

图 3-5 M-FPG_F1 折线图

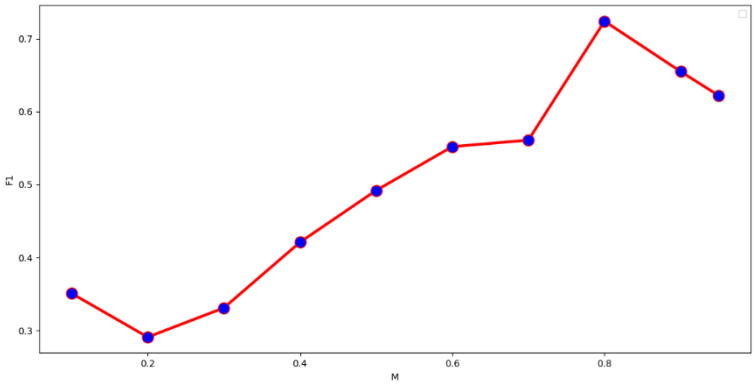

图 3-4 θ-FPG_F1 折线图

中国知网 https://www.cnki.net

3.6 本章小结

本章介绍了构建故障知识图谱的方法。故障知识图谱的构建也就是将历史故障数据沉淀为显式知识。本章首先介绍了运维事件、抽象运维事件、故障传播图和故障知识图谱等关键性定义，方便之后介绍故障知识图谱构建方案。构建故障知识图谱需要先从历史数据中生成事件，因此本章紧接着讲解了事件的生成方法。然后本章依次介绍了故障传播图和故障知识图谱的构建方法，最后，通过相关的实验验证了故障知识图谱构建方法的有效性。

中国知网 https://www.cnki.net

第四章 基于图卷积神经网络的故障根因分析

本章首先介绍故障根因分析的整体流程，然后介绍图相似度计算方法，接着详细介绍故障根因分析方法，之后为了便于理解本章会对一个故障溯源案例进行分析，然后进行了多组对比实验来验证本文故障根因分析方法的有效性，最后对本章内容进行小结。

4.1 故障根因分析方法概览

图 4-1 为故障根因分析方案概览，它能够利用多源异构数据构建故障知识图谱，然后使用故障根因分析方法得到故障根因事件，最后向用户显示故障的根本原因。故障根因分析方案分为两个部分，一部分是线下故障知识图谱的构建，一部分是线上故障的故障根因分析。本文使用系统的历史数据沉淀出故障知识图谱，作为经验信息为接下来的线上故障溯源提供帮助。一个故障知识图谱存放着一种故障的经验信息。

线下故障知识图谱构建主要利用了事件生成模块和故障知识图谱构建模块。本文通过事件生成模块从历史故障数据中生成事件，然后通过故障知识图谱构建模块先将事件构建成故障传播图，最终沉淀为故障知识图谱。

线上故障的故障根因分析主要利用了事件生成模块和故障根因分析模块。当系统出现故障时，事件生成模块会收集系统中的 Metric 和 Log 数据，并通过事件生成模块生成事件，然后故障根因分析模块会利用故障知识图谱构建模块中的故障传播图构建算法构建实时故障传播图，最后将实时故障传播图结合故障知识图谱进行分析得出根因事件。

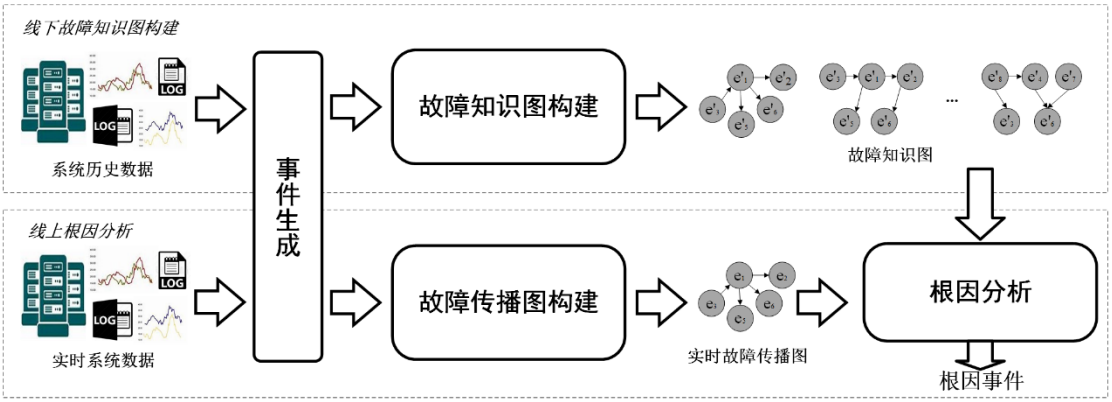

图 4-1 故障根因分析方案概览

4.2 图相似度计算方法

本节介绍图相似度计算方法，用来计算故障传播图和故障知识图谱之间的相似性。图相似度计算模型如图 4-2 所示。首先将图嵌入成计算机可以处理的向量表示和邻接矩阵，然后使用关系图卷积网络提取图的特征，接着进行池化和连接，最后输入多层感知机得到图相似度。下面详细介绍此模型。

图相似度计算需要同时考虑节点和边的相似度，而故障传播图和故障知识图谱的节点分别是事件和抽象事件，两种不同类型的节点是无法比较的，因此要统一节点类型。比较故障

中国知网 https://www.cnki.net

传播图和故障知识图谱本质上是比较两种故障的传播是否相似，不需要考虑具体时间和地点，只需要考虑事件发生的先后、地点类型和事件类型，也就是说只需要比较抽象事件的属性即可。因此计算故障传播图和故障知识图谱之间的相似性之前，需要将故障传播图的节点抽象化为抽象事件。

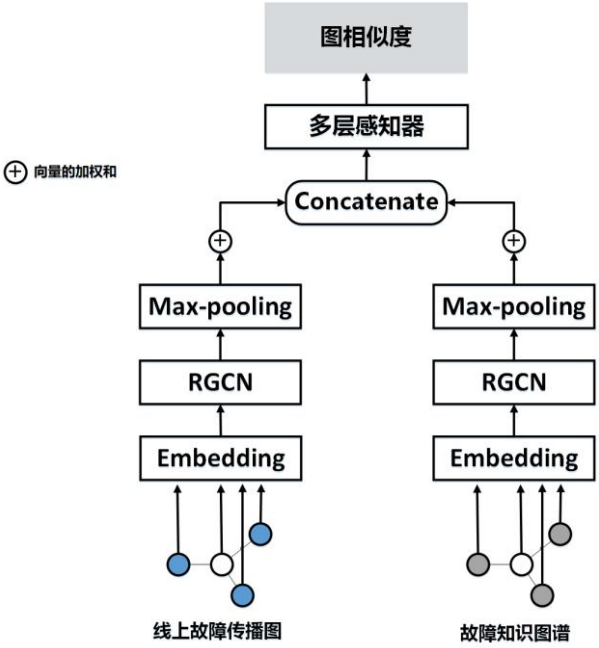

图 4-2 图相似度计算模型

计算机是无法处理抽象事件的，需要将抽象事件转换为计算机可以处理的向量表示形式。相似的抽象事件的向量表示在向量空间中应该更近，而不同抽象事件的向量表示在向量空间中更远，利于之后步骤中的图相似度计算模型更好地区分不同抽象事件从而获得更好的模型效果。目前可以使用的预训练模型有很多，如 Word2vec、Glove 和 Bert 等。Word2vec 不能解决同义词问题，如水果中的“苹果”和苹果公司的“苹果”，词向量表示是一样的，而实际上这两词的意思完全不一样。Bert 利用单词的上下文信息来做特征提取，根据上下文信息的不同动态调整词向量，解决一词多义的问题，而且拥有并行运算能力，因此本文采用 Bert 作为抽象事件的嵌入模型，如图 4-2 的 Embedding 层。

使用 Bert 模型将抽象事件转换为可由计算机处理的向量表示形式之后，可以将抽象事件的向量表示和图的邻接矩阵输入关系图卷积网络中抽取图的特征。有向图记为$\mathcal{G}=(\mathcal{V},\mathcal{E},\mathcal{R})$，其中节点$v_i\in\mathcal{V}$，边(关系) $(v_i,r,v_j)\in\mathcal{E}$，$r\in\mathcal{R}$是关系类型。$\mathcal{R}$包含正方向和反方向的关系，因此本文采用关系图卷积网络。本文使用公式(4-1)的传播模型计算图中节点v_i的前向更新。其中$h_i^{(l)}\in\mathbb{R}^{d^{(l)}}$表示图卷积神经网络的第$l$层中节点$v_i$的隐藏状态，而$d^{(l)}$是该层表示的维数。特别地，$h_i^{(0)}$是初始向量嵌入。$\mathcal{N}_i^r$表示所有和节点 i 具有关系$r\in\mathcal{R}$的邻居节点。$C_{i,r}$是归一化常数，设置为$|\mathcal{N}_i^r|$。$W_r^{(l)}$和$W_0^{(l)}$是图卷积神经网络的参数，可以对其进行训练。

$$h_i^{(l+1)}=\sigma\Big(\sum_{r\in\mathcal{R}}\sum_{j\in\mathcal{N}_i^r}\frac{1}{C_{i,r}}W_r^{(l)}h_j^{(l)}+W_0^{(l)}h_i^{(l)}\Big) \qquad (4-1)$$

中国知网 https://www.cnki.net

通过 R-GCN 获取图特征后，最大池化层作用于图向量矩阵，以获得图向量表示g_{FPG}和g_{FKG}。$g_{FPG}, g_{FKG} \in \mathbb{R}^{d^{(l_0)}}$表示图的向量表示，其维数与最后一个输出层相同。最后，使用公式(4-2)可以如下计算g_{FPG}和g_{FKG}之间的最终相似度。其中⊕表示向量的拼接。W_i, b_i表示第 i 个线性层的权重和偏差。感知机将返回维度为 2 的向量$pred_{FPG,FKG}$表示相似的可能性和不相似的可能性，也就是相似度。

$$pred_{FPG,FKG} = softmax(\sigma(W_1(\sigma(W_0(g_{FPG} \oplus g_{FKG}) + b_0)) + b_1)) \quad (4-2)$$

最后，使用公式（4-3）可以找到和故障传播图最相似的故障知识图谱。

$$i = \underset{i \in |FKBs|}{\arg max}\,(pred_{FPG,i}[1]) \quad (4-3)$$

4.3 基于图卷积神经网络的故障根因分析方法

系统出现故障事件 f 时，故障传播图构建模块会通过故障传播图构建算法构建 FPG，然后使用图 4-2 中的基于 R-GCN 的图相似度计算模型计算 FPG 和各个 FKG 之间的相似度，找出与 FPG 最相似的 FKG。此 FKG 和 FPG 在事件之间具有相同的逻辑关系，因此它们应该具有最高的相似性，所以他们更可能是相同的故障，具有相似的根因。 所以故障根因分析模块选取此 FKG 在 3.4.2 中标注的根因事件类型作为故障事件 f 的根因事件类型。此过程如图 4-3 所示。

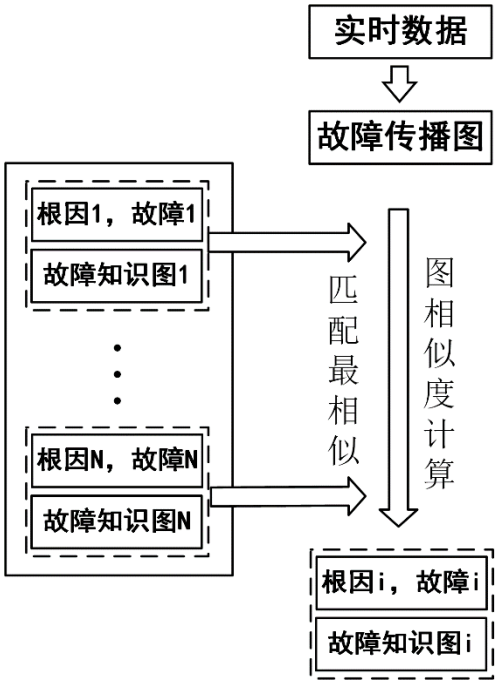

图 4-3 根因分析方法

由于此处返回的是根因事件类型，需要对应到具体的根因事件。公式(4-4)是具体根因事件s_{root}的计算公式。其中S是根据此根因事件类型统计实时数据中所有为该事件类型的事件集合。$N_t(s)$函数返回事件s按s与f发生时间差顺序在集合S中的排名，$N_d(s)$函数返回事件s按s与f发生设备距离顺序在集合S中的排名，W_1，W_2分别为两个超参数权重。由此公式就可以得到故障事件f的根因事件s_{root}，故障根因分析完成。

中国知网 https://www.cnki.net

$$s_{root} = \underset{s \in S}{\operatorname{argmin}}(W_1 * N_t(s) + \ W_2 * N_d(s)) \qquad (4-4)$$

4.4 RCA 案例分析

本节将使用一个实际案例来展示本文提出的故障根因分析方法。用户在 2019-12-12 03:45:48 购买火车票时，火车票销售服务出现故障。 此故障的根本原因是 Pod 配置错误。故障触发过程为：

1、用户预订火车票，系统调用旅行微服务。

2、旅行微服务调用 Ticketinfo 微服务以查询火车票信息。

3、Ticketinfo 微服务调用基本微服务以查询基本用户信息。

4、由于配置错误，基本微服务所在的 Pod 会继续重新启动。结果，Ticketinfo 微服务无法访问基本微服务，并且发生 Spring http 服务器错误。然后，调用票证信息微服务的旅行微服务一个接一个地发生错误。

故障根因分析系统首先在故障发生前十分钟（可以根据领域专家设置时间窗口大小）收集分布式系统的 Metric，日志和 Tracing，自动构建在线故障传播图。然后，图相似度计算模块通过图相似度模型获得与在线故障传播图最相似的故障知识图谱。 知识图谱由领域专家标记，其中包含根本原因的类型和故障传播路径。根据公式(4-4)，我们最终可以得出故障的根本原因和故障传播路径。最终故障根因分析系统将结果可视化，如图 4-4 所示。从图中可以看到失败的根本原因是 Pod 配置错误。故障根因分析系统通常会返回 Top3 故障的根本原因，运维人员可以参考此结果来分析故障。

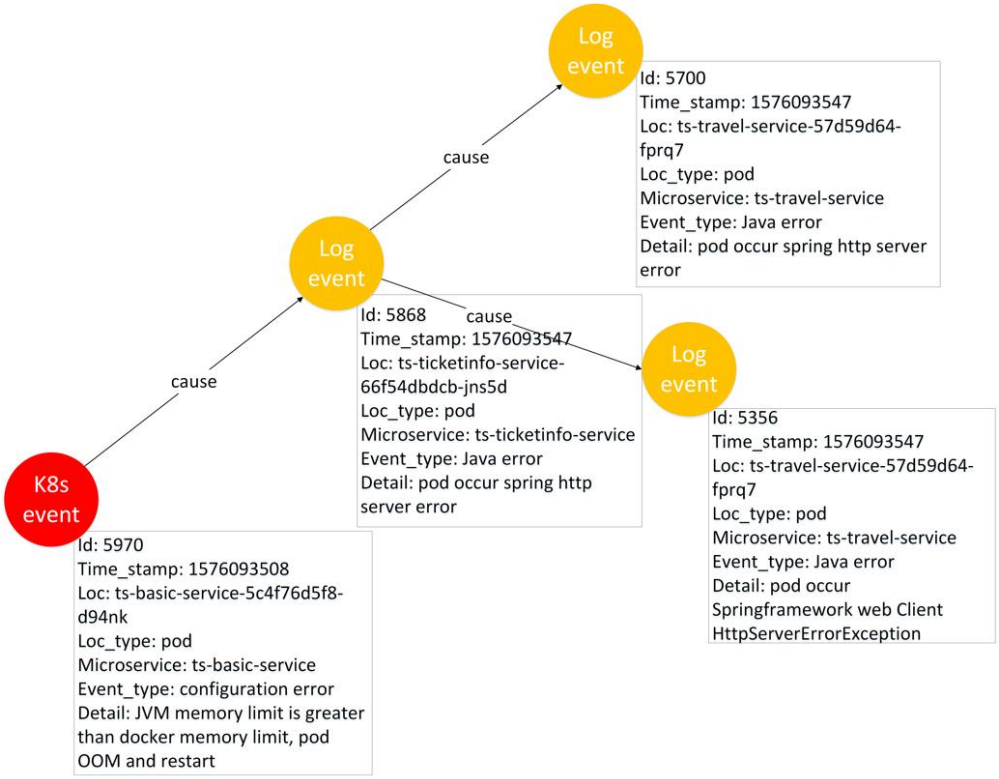

图 4-4 故障根因分析实例

中国知网 https://www.cnki.net

4.5 实验

4.5.1 数据集与评测指标

本节数据集采用 3.5.1 中的数据。本文实验主要对故障根因分析方法做评测，评测公式如公式(4-5)(4-6)(4-7)。

$$Precision' = \frac{|R_{correct}{}'|}{|R'|} \tag{4-5}$$

$$Recall' = \frac{|R_{correct}{}'|}{|R_{label}{}'|} \tag{4-6}$$

$$F1' = \frac{2 * Precision' * Recall'}{Precision' + Recall'} \tag{4-7}$$

其中R'表示所有故障根因分析得到的故障根因事件集合，$|R'|$表示故障根因分析得到的故障根因事件数目，$R_{correct}{}'$表示故障根因分析得到的正确的故障根因事件集合，$|R_{correct}{}'|$表示故障根因分析得到的正确的故障根因事件数目。$R_{label}{}'$表示标注故障根因事件集合，$|R_{label}{}'|$表示标注故障根因事件数目。

4.5.2 实验结果

本节将展示故障根因分析模型在根因分析效果，同时对比其他四种故障根因分析模型：Codebook[4]、贝叶斯网络[14]、MicroCause[5]和系统图[6]方法。从表 4-1 可以看到，本文的根因分析模型较现有的方法在 Precision、Recall 和 F1 上均有更好的效果。

Codebook 是一种基于编码的故障根因分析方法，它把事件分解为症状和问题，并根据历史数据自动构建它们之间的关联，以此达到找到根因的目的。Codebook 仅仅通过将故障和根因联系起来，没有使用显式的故障传播路径的知识，因此此模型在根因分析精确率、召回率和 F1 值都不如本文方法。

贝叶斯网络是一种基于概率图的故障根因分析方法，它把故障或者异常作为随机变量，然后通过使用历史数据建立概率模型计算相互关联的变量之间的转移概率，最后经过推理得到故障的根因。MicroCause 是一种基于 Metric 依赖图的故障根因分析方法，它通过使用历史数据挖掘 Metric 之间的依赖关系，得到故障在 Metric 上的传播路径从而完成根因分析。系统图的方法是一种基于组件依赖图的故障根因分析方法，它通过使用历史数据挖掘组件之间的依赖关系，得到故障在组件上的传播路径从而完成根因分析。虽然这三种方法都使用了显式的故障传播路径的知识，但是这些方法对故障传播路径的挖掘不够充分。贝叶斯网络只考虑到了异常事件的传播，忽略了正常事件也可能传播故障，而且它忽略了事件发生的地点，相同的事件在不同的设备发生，可能引发不同的故障。MicroCause 只考虑到了故障在不同 Metric 之间传播，忽略了故障在同一 Metric 的传播。设备图的方法只考虑到故障在不同组件之间的传播，忽略了故障在设备内部的传播。这三种方法使用的故障传播路径都不完整，而本文使用故障知识图谱具有事件级别的故障传播路径，可以克服上诉问题。从表 4-1 可以看到，三种方法的根因分析精确率、召回率和 F1 值都不如本文方法。

中国知网 https://www.cnki.net

表 4-1 RCA 对比实验结果

数据集	方法	Presicion	Recall	F1
Train-ticket	本文方法	0.867	0.825	**0.845**
	Codebook	0.702	0.698	0.700
	贝叶斯网络	0.444	0.569	0.499
	系统图	0.613	0.624	0.619
	MicroCause	0.620	0.510	0.560
Sock-shop	本文方法	0.826	0.815	**0.820**
	Codebook	0.701	0.694	0.697
	贝叶斯网络	0.418	0.504	0.457
	系统图	0.614	0.606	0.610
	MicroCause	0.549	0.518	0.533

4.6 本章小结

本章介绍了如何利用故障知识图谱进行故障根因分析。首先从整体上介绍了整个根因分析流程，主要包括构建故障知识图谱、构建在线传播图和根因分析，然后依次介绍了图相似度计算方法，用于计算在线故障传播图和故障知识图谱的相似度，接着介绍了故障根因分析方法， 之后为了便于理解本章对一个故障溯源案例进行了分析，然后进行了多组对比实验来验证本文故障根因分析方法的有效性。

中国知网 https://www.cnki.net

第五章 故障根因分析系统的设计与实现

本章对故障根因分析的系统进行详尽的需求分析，并根据第三章的故障知识图谱构建方法和第四章的故障根因分析方法，设计并实现了一个故障根因分析系统。

5.1 故障根因分析系统需求分析

故障根因分析系统的需求分析包括功能需求和非功能需求。功能需求是指故障根因分析系统应当实现的功能，而非功能需求是指故障根因分析系统应当满足的除功能需求之外的需求。

5.1.1 功能需求

针对故障根因分析系统进行的功能性调研表明，故障根因分析系统从功能点上分为三大类：设备搜索功能，异常发现功能，故障根因分析功能。系统用例图如图 5-1 所示：

(1) 设备搜索

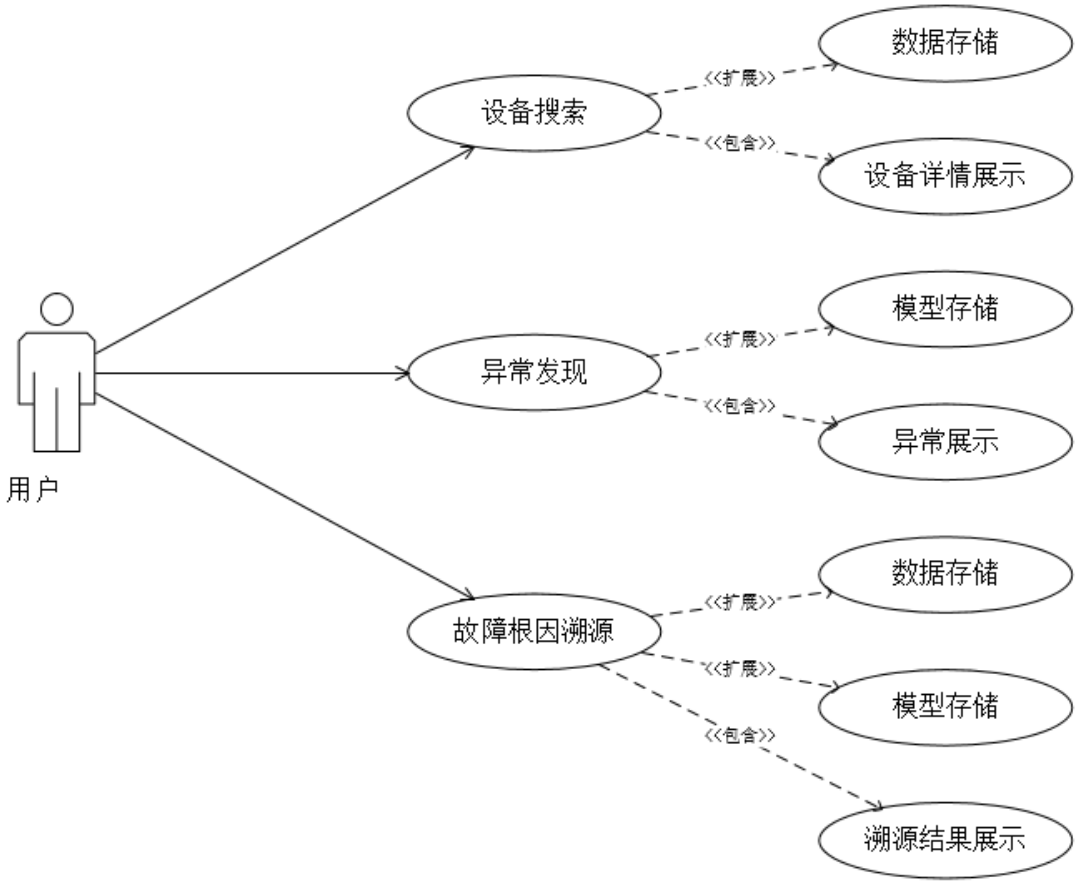

图 5-1 系统用例图

本系统的用户大多是后台服务器的运维人员，服务器运维人员需要及时了解服务器整体的状态。随着虚拟化技术的发展，服务器内可容纳几十上百个容器。运维人员平均需要监控几十个服务器，同时每个服务器内可能有几十上百个容器，搜索服务器详细设备属性信息的接口和搜索容器详细设备属性信息的接口不同，搜索设备属性信息的接口和搜索监控信息的

中国知网 https://www.cnki.net

接口也不同，运维人员需要频繁调用不同接口，使用不同搜索软件才能全方面的了解到某个设备的状态，这个过程相当繁琐。因此，运维人员急需一种可以全方面查看整个服务器集群状态信息的设备搜索系统。此设备搜索系统需要整合多源、异构数据，融合服务器集群中搜索设备属性信息的各个接口，同时存储服务器集群中设备属性信息，支持更新服务器设备属性信息。在搜索方面，此设备搜索系统需要支持搜索设备的详细属性信息、设备监控信息以及告警，并且支持精确搜索和模糊搜索，同时可以查看服务器集群的拓扑结构。满足上面的数据存储和设备详情展示是本系统的需求。

(2) 异常发现

在真实的运维场景中，运维人员不可能实时监控所有的设备。当运维人员发现服务器集群出现某些故障时，这些故障可能已经出现了很久，甚至可能错过了最佳修复时间，造成了不可挽回的损失。所以本系统需要拥有异常发现的能力，当发现服务器集群中出现故障时，要及时反馈给运维人员，保证故障得到及时处理。服务器集群中的异常可能会出现在各个层面，设备的监控曲线中可能蕴含异常，设备的日志记录中也可能记录着异常，一些监控平台上的告警信息也是一种异常。所以异常发现模块需要有多个模型，从不同的监控曲线、日志记录和监控平台告警中发现异常，可能需要数十个模型，所以需要模型存储的功能需求。异常发现后需要及时通知运维人员，同时将异常输入到故障根因分析模块中进行故障根因分析，因此这里需要一个异常发现的异常展示功能。满足上面的模型存储和异常展示功能是本系统的需求。

(3) 故障根因分析

当运维人员发现故障之后，他们需要寻找和这个故障相关的设备，查看设备的详细属性信息以及监控记录，这个过程是很繁琐的，复杂的故障通常需要 1-2 个小时，严重影响了服务器集群的正常运行以及业务的办理，这可能会给公司带来巨大的损失。因此本系统需要拥有故障根因分析的能力，在发现故障后可以自动进行故障根因分析，返回给用户可能的故障根因，同时返回故障传播链，让运维人员了解故障传播过程，从而更好的解决故障。系统需要保留历史故障根因分析记录，方便运维人员查看，因此需要数据存储模块。为了保证自动溯源效果，系统使用了机器学习模型，因此需要模型存储模块。同时，为了更立体地展示故障根因分析结果，系统需要从事件-设备两个层次展示故障的传播过程。系统不能保证得分最高的故障根因分析结果就是真实场景的故障根因，所以系统需要返回 TOP3 的故障根因分析结果供运维人员查看。满足上面的模型存储、数据存储和溯源结果展示功能是本系统的需求。

5.1.2 非功能需求

系统在故障根因分析过程中，相比于其他故障根因分析方法，异常发现要迅速，当分布式系统中有异常发生时，系统需要快速的找到异常并将异常反映给运维人员，应当保持秒级的响应时间。

中国知网 https://www.cnki.net

5.2 系统总体设计

系统需求分析介绍了设计故障根因分析系统的主要目的、系统需要实现的功能需求以及非功能需求，本节根据系统需求分析设计系统的总体架构。

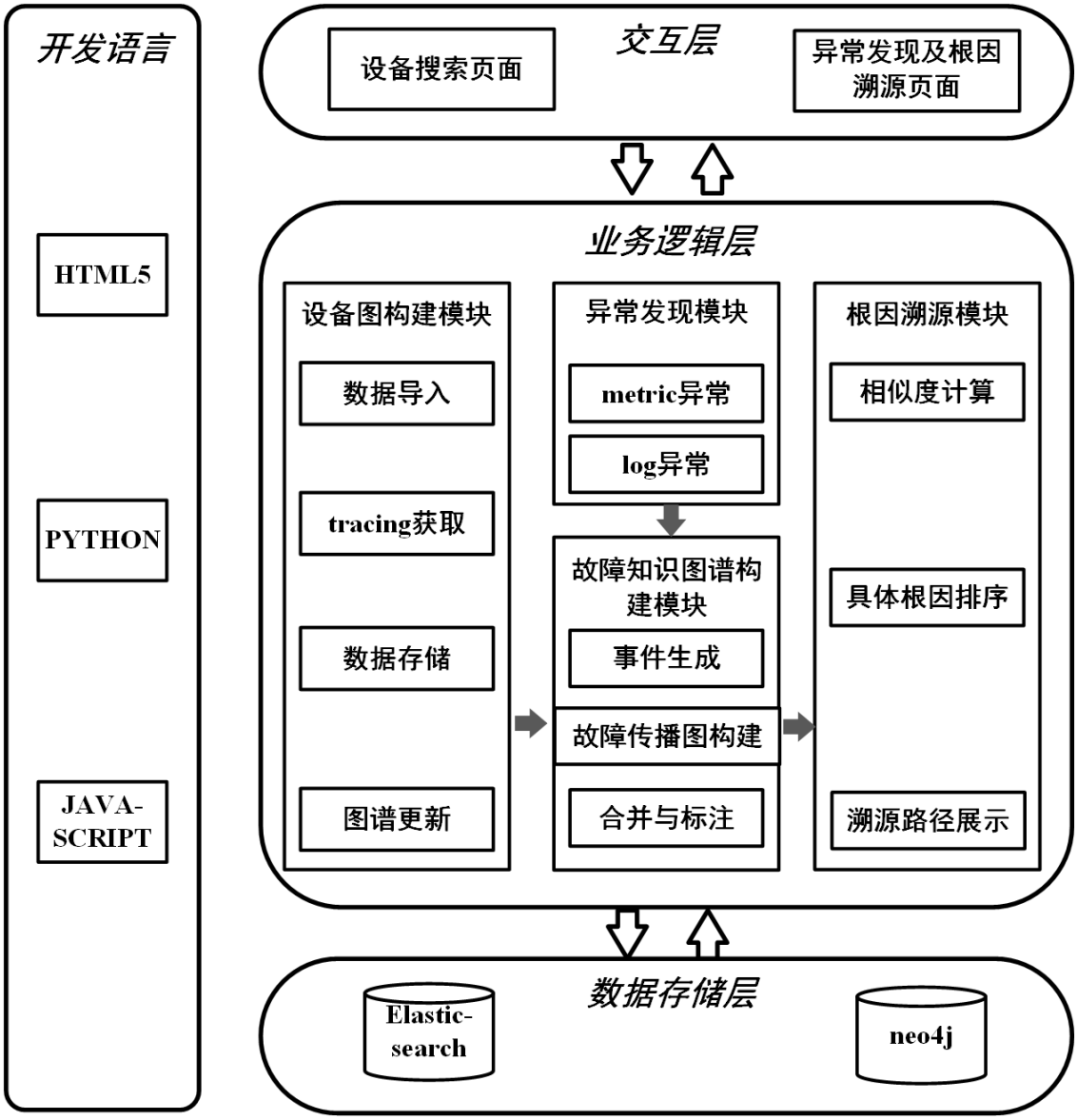

图 5-2 故障根因分析系统框图

如图 5-2 所示，故障根因分析系统主要分为三层：交互层，业务逻辑层以及数据存储层。交互层负责整个系统的输入和输出，负责将用户的操作传递给系统，并将系统的信息传递给用户，交互层由两个页面视图构成，一是设备搜索页面，在该页面输入设备 ID 搜索该设备的详细属性、相关拓扑结构以及监控信息；二是异常发现和故障根因分析展示视图，在该界面展示当前发现的异常以及故障的根因。业务逻辑层负责整个系统的业务逻辑，主要由设备图谱构建模块，异常发现模块，故障知识图谱构建模块和故障根因分析模块四个模块构成。下面将对四个模块一一介绍。

中国知网 https://www.cnki.net

(1) 设备图谱构建模块

设备图谱构建模块主要负责构建集群的设备图谱。由于提前构建好了集群的设备图谱，交互层只需访问设备图谱就可以快速搜索集群中相关设备的信息。设备图谱构建模块分为数据导入、tracing 获取、数据存储和图谱更新四个部分。首先需要获取集群中的设备信息，然后获取集群的 tracing 信息，将这些数据存储起来，然后使用这些数据构建设备图谱。由于集群中的设备是会变动的，所以这里的设备图谱需要实时更新。但是有时候运维人员可能需要 1 小时前的设备图谱，所以本文采用将时间属性加入设备图谱的方式对设备图谱进行更新，可以保存设备图谱的历史信息。

(2) 异常发现模块

异常发现模块主要负责发现集群中的异常。此模块主要有日志异常发现和 metric 异常发现两个部分。日志异常发现主要是通过日志检测算法或者模板对日志进行筛选，找到集群中异常的日志，然后处理后返回。Metric 异常发现主要是通过 metric 异常检测算法或者模板发现集群中的 metric 异常，然后处理后返回。

(3) 故障知识图谱构建模块

故障知识图谱构建模块主要负责故障知识图谱的构建。此模块主要有事件生成、故障传播图构建和合并与标注三个部分。此模块主要的功能是从历史数据中挖掘出故障的传播路径信息，然后作为经验保存下来。首先从历史数据中通过聚类算法、异常发现算法和模板的方式生成事件，然后使用 2.3 中的故障传播图构建算法构建故障传播图，最后通过 2.4 的合并和标注算法构建故障知识图谱并进行标注。

(4) 故障根因分析模块

故障根因分析模块主要负责找到故障的根本原因。此模块主要有相似度计算、具体根因排序和溯源路径展示三个部分。故障发生时，此模块会构建一张故障传播图，然后和故障知识图谱进行相似度计算，找到最相似的一张故障知识图谱，将此故障知识图谱中的根因类型作为此次故障的根因类型，然后通过事件的发生时间和发生距离得到具体根因排序。并将故障知识图谱中标注的故障路径作为此次溯源的故障路径，最后通过交互层进行溯源路径展示。数据存储层主要负责存储数据，本系统需要存储的数据包括设备图谱、故障知识图谱以及原始数据。考虑到本文构建的设备图谱和故障知识图谱都是图结构，而 neo4j 是图数据库，适合图结构的数据，对长程关系的查询速度快，擅于发现隐藏的关系，因此数据存储层选取 neo4j 存储设备图谱和故障知识图谱。对于原始数据，本文使用阿里云监控接口收集服务器级别的日志和监控信息，使用普罗米修斯收集容器和 pod 中的日志和监控信息，使用 weave scope 采集容器级别的拓扑结构，最后通过获取微服务的 tracing 获取微服务级别的拓扑结构。由于这些原始数据规模庞大，多为时序数据和日志数据，而 Elasticsearch 是一个分布式、高扩展、高实时的搜索与数据分析引擎，能很方便的使大量数据具有搜索、分析和探索的能力。所以数据存储层采用 Elasticsearch 存储原始数据。

中国知网 https://www.cnki.net

5.2.1 设备图谱构建模块概要设计

此模块主要功能是将集群设备的详细信息和拓扑结构存储成图结构。此模块主要需要设计并实现数据的收集、设备图谱节点和边的构建、设备图谱更新三个部分。设备图谱的节点是设备，节点属性是设备属性，边是设备之间的连接或者虚拟设备之间信息交互，因此数据的收集包括设备属性、设备监控数据、设备连接性和 tracing。导入数据后进行设备图谱的构建，利用设备 id 和设备属性生成节点及其属性，然后利用设备连接性和 tracing 构建节点和节点之间的边。由于集群是不断变动的，因此需要对设备图谱进行更新，而且运维人员需要查找历史的集群结构，所以更新需要版本回退功能，本文采用将节点和关系加入时间属性的方式完成此功能。

5.2.2 异常发现模块概要设计

此模块的主要功能是发现集群中的异常。此模块的数据主要包括集群中的 metric、集群中的日志和集群中出现的平台告警。集群中出现的平台告警可以直接当作异常返回。对于 metric 数据和日志数据需要使用两种方法从数据中获取异常。Metric 数据可以看作时序数据，对于时序数据可以采用异常检测算法识别出异常点，然后采用模板的方式获得异常事件返回异常。对于日志数据，采用聚类获取相似日志模式，然后使用模板生成异常事件返回异常。此模块的关键是针对两类数据的两种不同的异常发现方法的实现。

5.2.3 故障知识图谱构建模块概要设计

此模块的主要功能是构建故障知识图谱。此模块主要需要设计并实现历史数据的收集、事件生成、故障传播图构建、合并和标注四个部分。故障根因分析模块是冷启动的，需要事先构建好故障知识图谱。故障知识图谱作为一种从历史数据中挖掘的显式知识，其数据应当是集群的历史数据。历史数据包括集群中的 metric、集群中的日志和集群中出现的平台告警。收集好的历史数据存放在 Elastic- search 中，便于之后查找。然后需要设计并实现事件生成功能，此功能从历史数据中通过聚类算法、异常发现算法和模板的方式生成事件。接下来需要将 3.3 中的故障传播图构建算法进行封装，实现故障传播图构建功能，最后设计并实现 3.4 的合并和标注算法构建故障知识图谱并进行标注。

5.2.4 根因分析模块概要设计

此模块的主要功能是找到集群中故障的根因。此模块主要需要设计并实现相似度计算、具体根因排序和溯源路径展示三个部分。首先需要实现相似度计算功能，此功能使用 python 的 pytorch 框架实现相似度计算中的 R-GCN 模型，从而实现相似度计算。然后将 4.3 中的公式（4-4）使用 python 实现并封装完成具体根因排序功能。最后将故障知识图谱中标注的故障路径作为此次溯源的故障路径上传至交互层实现溯源路径展示功能。

中国知网 https://www.cnki.net

5.3 系统详细设计与实现

5.3.1 设备图谱构建模块详细设计与实现

首先设计设备图谱的本体。经过对集群结构的调研和询问相关领域专家后，本文共设计了 11 种设备本体，315 种本体属性，具体本体统计如表 5-1，部分本体结构如图 5-3。

表 5-1 设备图谱本体统计

本体名称	属性数量
集群（Kubernetes）	11
云服务器（ECS）	113
云数据库（RDS）	43
负载均衡（SLB）	7
服务器主进程	10
Kubernetes Pod	12
容器（container）	16
集群服务（Service）	13
虚拟私有网络（VPC）	20
虚拟交换机（VSswitch）	8
云监控面板（Dashboard）	58
集群应用部署（Deployment）	4

然后使用云监控、Weave Scope、Jaeger 和普罗米修斯收集集群中的设备数据和拓扑结构数据。阿里云监控接口可以收集服务器级别的日志和监控信息，普罗米修斯可以收集容器和

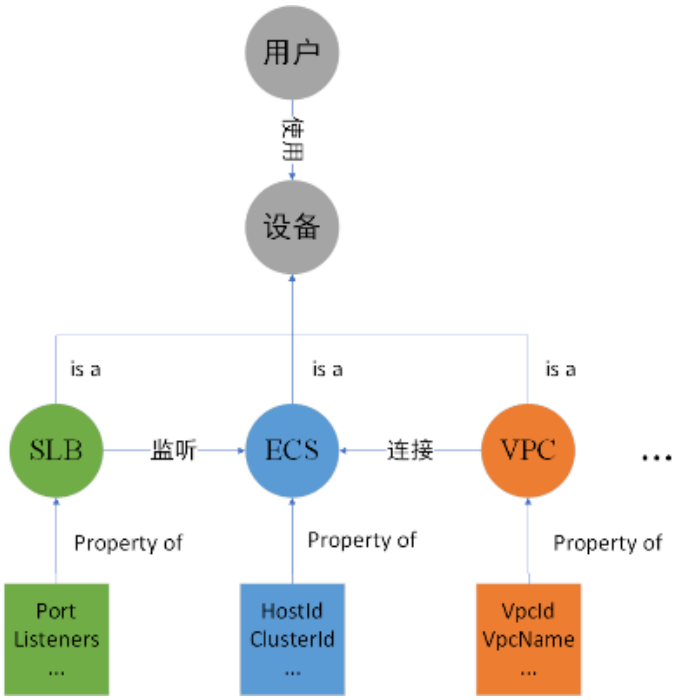

图 5-3 部分设备本体结构

中国知网 https://www.cnki.net

Pod 中的日志和监控信息，Weave Scope 可以采集容器级别的拓扑结构，Jaeger 可以获取微服务的 Tracing 从而获取微服务级别的拓扑结构。

收集设备数据完成后，利用设计好的本体结构，依次生成相应的节点和边，然后存入图数据库 Neo4j 中。集群是不断变动的，设备图谱也是变动的，因此需要对设备图谱进行更新。除了实现基本的更新功能外，还需要实现版本回退功能。本文将时间属性加入设备图谱中节点和关系。设备图谱的每个节点和边在创建时都会添加一条创建时间属性，每次删除节点和边时不会直接删除，而是添加一条删除时间属性，这样就可以查看每个节点和边的生命周期，从而确定每个节点和边在某一确定时刻是否存在，从而实现版本回退功能。图 5-4 显示了设备图谱中某服务器节点的生命周期。最后故障根因分析系统每过 1 分钟会重新获取集群中设备信息，集群中的新的设备会在设备图谱上会被创建，集群中消失的设备会在设备图谱上“删除”，完成更新功能。


```
{
  "HostName":"iZ2zecsx6bxpn6lpwzveyjZ",
  "OSName":"CentOS  7.6 64位",
  "Memory":"65536",
  "Cpu":"8",
  "ImageId":"centos_7_06_64_20G_alibase_20190619.vhd",
  "CreateTime": "2019-09-10T05:23Z",
  "EndTime": "-",
  "exist":true
}
```


图 5-5 某服务器节点的生命周期

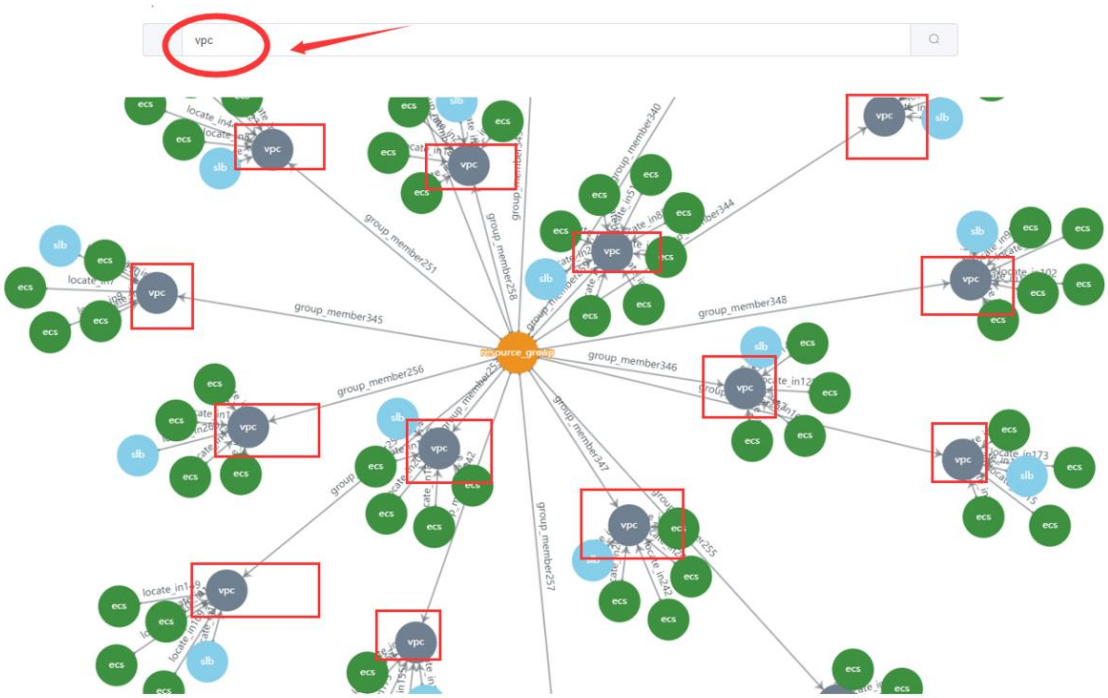

图 5-4 查询某类设备及其周边节点

中国知网 https://www.cnki.net

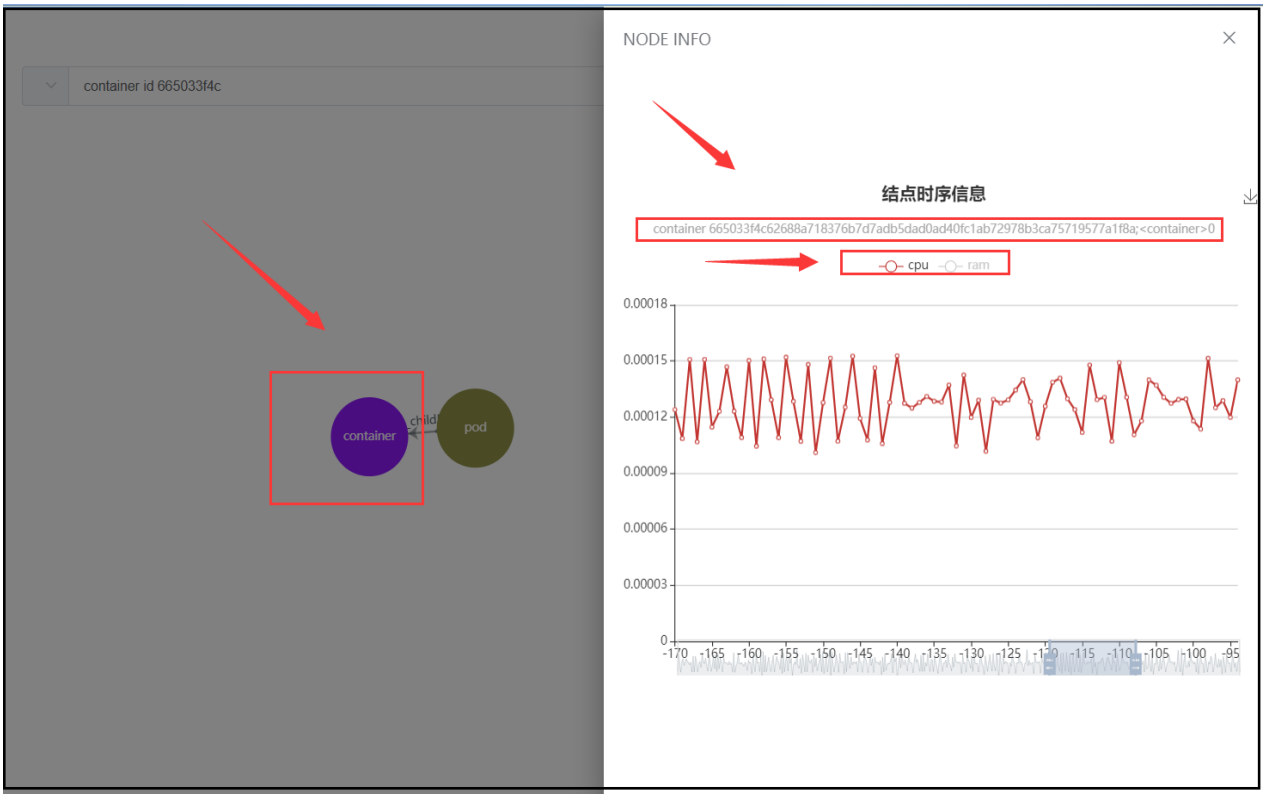

图 5-6 在设备图谱中查询设备的监控信息

最后交互模块在收到运维人员的查询命令后会从 Neo4j 中查询相应的命令，获取数据返回至前端，经过网页渲染生成可视化的设备图谱。图 5-5 和图 5-6 分展示了查询某类设备及其周边节点和查询设备的监控信息两个功能。

5.3.2 异常发现模块详细设计与实现

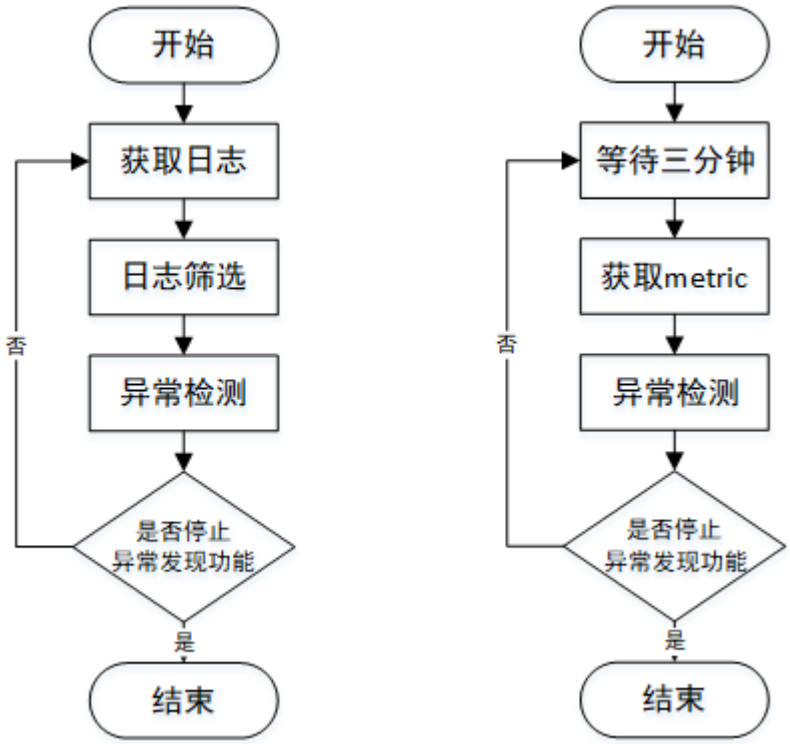

图 5-7 日志数据异常检测流程图（左）和 metric 数据异常检测流程图（右）

中国知网 https://www.cnki.net

此模块一旦开启就会一直检测集群中的异常，直到运维人员关闭此功能。检测分为定时检测和推送检测两种。定时检测是隔一段时间对历史数据进行异常检测，此方法适用于稀疏型数据；推送检测是只要有新数据产生就对此数据进行异常检测，此方法适用于密集型数据。

Metric 数据属于时序数据，单个点无法进行异常检测，需要累积一定的时序数据才能进行检测，因此本模块采用定时检测方法，时间窗口设为 3 分钟。日志数据属于密集型数据，集群一分钟可能产生数万条日志，因此本模块采用推送检测方法。由于日志数量巨大，在检测前需要使用 TF-IDF 和周期性方法删除噪声日志。TF-IDF 常用于自然语言处理当中，用于判断一个词语对于文本的重要程度，应用在本模块的主要的思想是如果一个日志无论在什么时候都经常发生，那么这个日志可以视为噪声日志。而且如果一个日志周期性出现，这就说明此日志也是噪声日志。日志数据异常检测流程图和 Metric 数据异常检测流程图如图 5-7 所示。最后从两种数据中发现的异常都将传入交互层，由交互层展示给运维人员查看。

5.3.3 故障知识图谱构建模块详细设计与实现

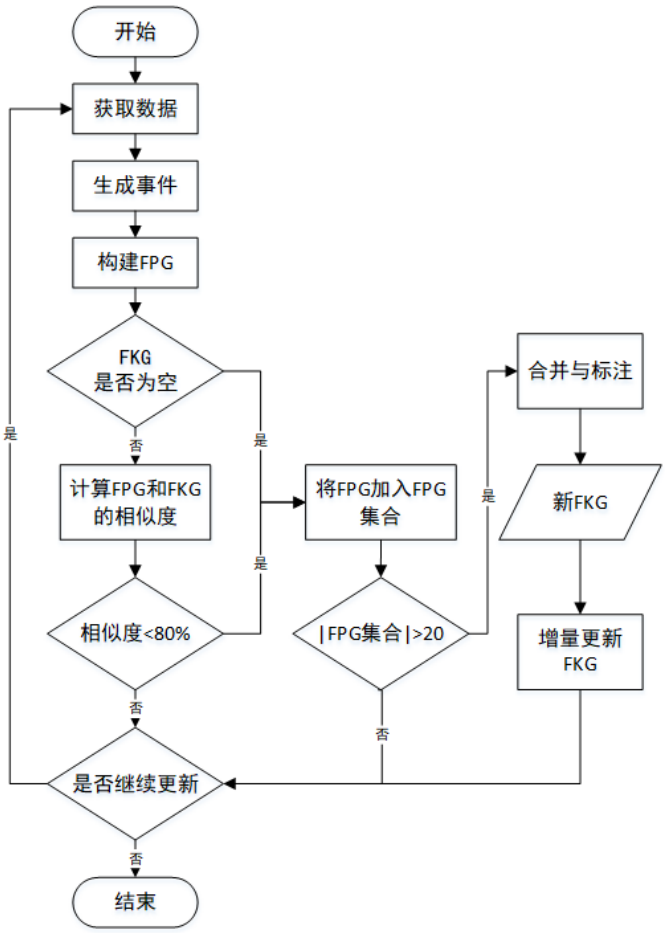

图 5-8 故障知识图谱构建与更新流程图

此模块首先需要收集集群中的历史故障数据，然后将历史数据分为两部分，一部分用于计算事件的置信度，另一部分用于构建故障知识图谱。历史数据包括集群中的 Metric、集群中的日志和集群中出现的平台告警。利用 3.2 中的事件生成方法从历史数据中生成事件，然后使用 3.3 中的故障传播图构建算法构建故障传播图，最后编程实现 3.4.2 的合并和标注算法构建故障知识图谱并进行标注。

中国知网 https://www.cnki.net

由于会有故障知识图谱中未出现的故障出现，需要对故障知识图谱进行增量更新。更新时首先会根据数据会生成新的故障传播图，如果和历史故障知识图谱的相似度都很低，那么历史故障传播图集合中。当历史故障传播图集合数目大于一定阈值时，对此集合中的故障传播图进行合并与标注，生成新的故障知识图谱，完成故障知识图谱的更新。故障知识图谱构建与更新流程图如图 5-8。

5.3.4 根因分析模块详细设计与实现

此模块首先需要实现相似度计算功能。相似度计算模型使用的是 4.2 章中的 R-GCN 模型，具体实现使用了 Python 的 Pytorch 框架。然后对于新出现的故障，收集该故障前 10 分钟的数据并构建故障传播图，将此故障传播图和故障知识图谱进行相似度计算，选择最相似的

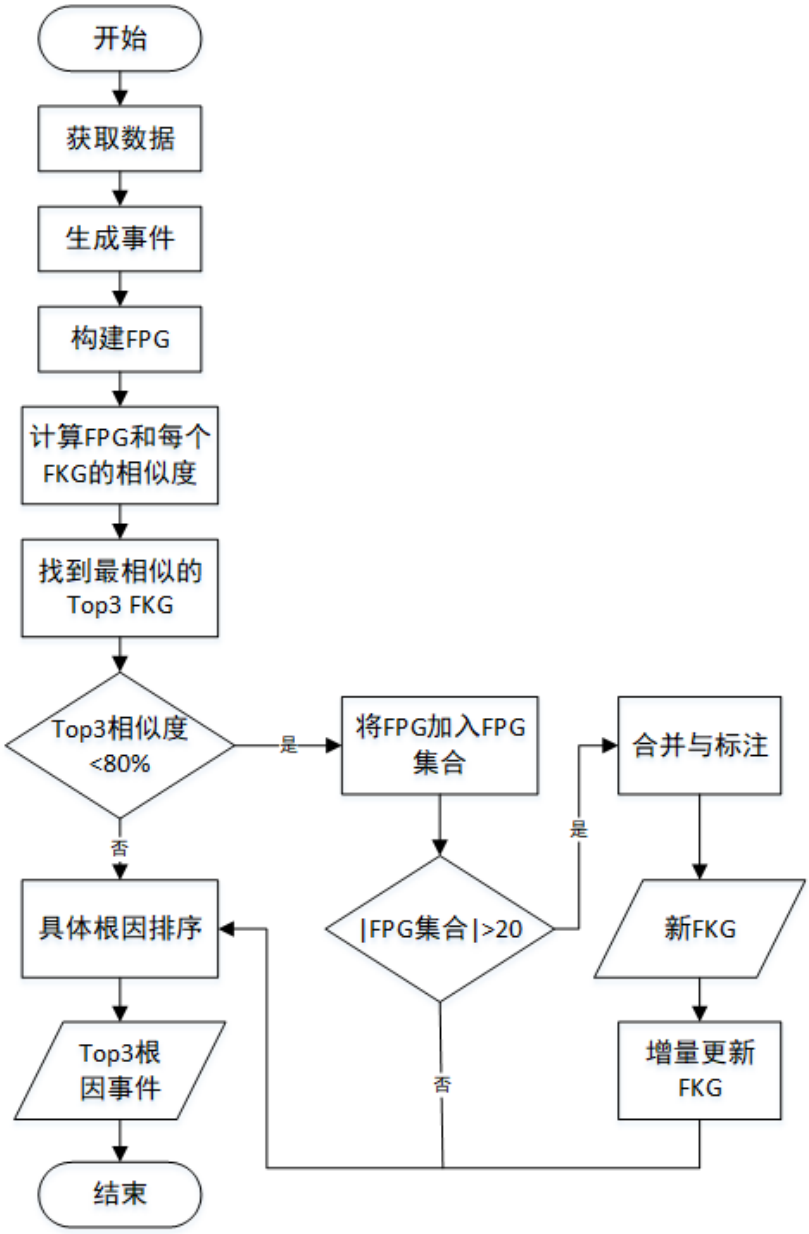

图 5-9 故障根因分析流程图

Top3 故障知识图谱的标注根因类型作为本次故障的 Top3 根因类型，然后将 4.3 章中的公式(4-4)使用 Python 实现并封装完成具体根因排序功能，利用根因类型找到具体根因事件，返回

中国知网 https://www.cnki.net

Top3 的根因事件。最后将故障知识图谱中标注的故障路径作为此次溯源的故障路径上传至交互层实现溯源路径展示功能。故障根因分析流程图如图 5-9 所示。

5.4 系统测试

本章根据第四章的需求分析中系统的功能需求和非功能需求，采用 Python 标准库中自带的单元测试框架 Unittest 作为测试框架来进行系统测试。测试的内容有两个部分：设备查询测试，故障根因分析测试。

5.4.1 系统测试环境

本文中系统测试环境分为硬件环境和软件环境，具体信息如表 5-2 所示。

表 5-2 系统测试实验环境

环境类型	环境内容	环境参数
硬件环境	集群	集群类型：Kubernetes 托管版 版本：1.12.6-aliyun.1 节点个数：5 节点操作系统：CentOS 7.6 64 位 节点 CPU： 8 核 节点内存： 16 GiB
	服务器	操作系统：Ubuntu 16.04 处理器：Intel(R) Xeon(R) CPU E5-2630 v4 @2.20GHz 内存：256GB 显卡：Nvidia RTX 2080Ti 显存：11GB 硬盘：2TB
软件环境	Elasticsearch	版本号 6.3.1
	Neo4j	版本号 3.5.6
	python	版本号 3.7.3

5.4.2 设备查询测试

运维人员可以用此功能查询集群中设备的属性、监控和拓扑结构信息。测试将会对 5 种搜索功能、一种更新功能进行测试，然后检验搜索结果，设备查询的测试用例如表 5-3 所示。

中国知网 https://www.cnki.net

表 5-3 设备查询功能测试用例

内容项	描述
测试目的	测试设备查询功能
前置条件	用户进入查询首页
用例设计	1．输入查询语句“vpc id k1aexj”，模糊查询 id 为 k1aexj 的 vpc 节点，点击查询按钮。 2．输入查询语句“vpc”，查询所有“vpc”类别的设备节点，点击查询按钮。 3．输入查询语句“slb balance”，查询 slb 及其周边节点，点击查询按钮。 4．输入查询语句“vpc slb ecs”，查询特定路径的所有结点，点击查询按钮。 5．双击某 vpc 节点，查询该节点的监控信息。 6．删除和增加集群中的节点。
预期结果	1．返回查询结果，查询结果是 id 前缀为 k1aexj 的 vpc 节点。 2．返回查询结果，查询结果是所有 vpc 类别的设备节点。 3．返回查询结果，查询结果是所有 slb 及其周边节点。 4．返回查询结果，查询结果是路径为“vpc slb ecs”的所有结点。 5．网页右侧出现此 vpc 节点的监控信息。 6．设备图谱中更新了集群中删除和增加的节点。
测试结果	六个用例经过测试与预期结果相符合
状态	通过

设备查询首页输入查询语句的界面和查询结果如图 5-10，图 5-11 所示。由于查询结果太多，本文只截取了部分设备查询的详细界面。

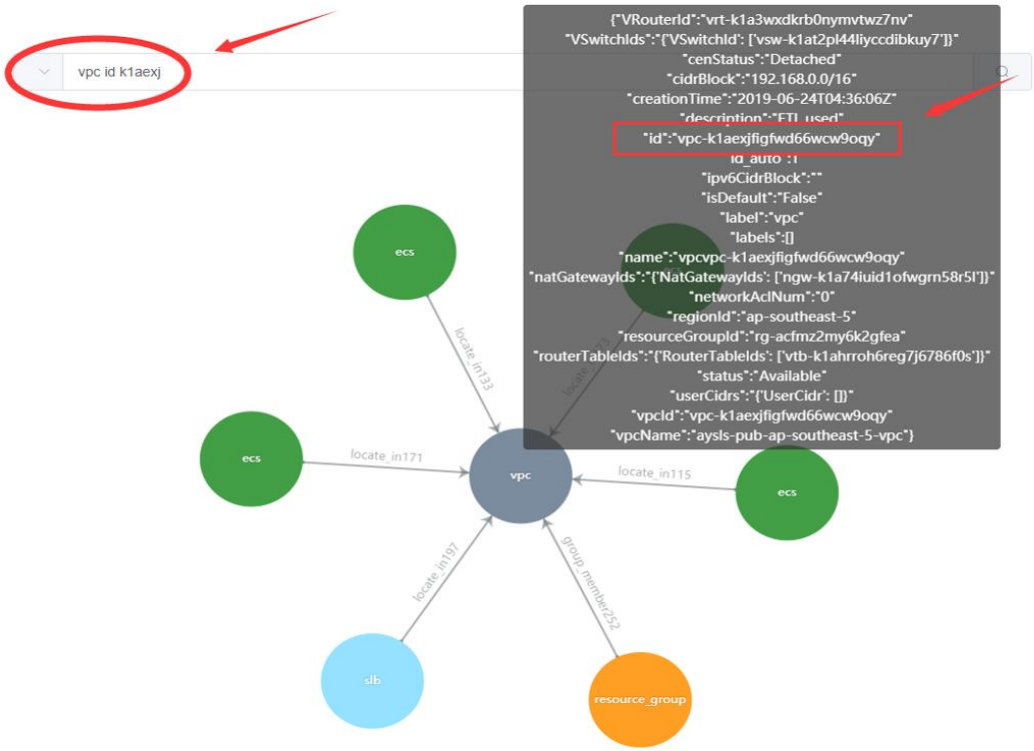

图 5-10 模糊查询特定属性的节点

中国知网 https://www.cnki.net

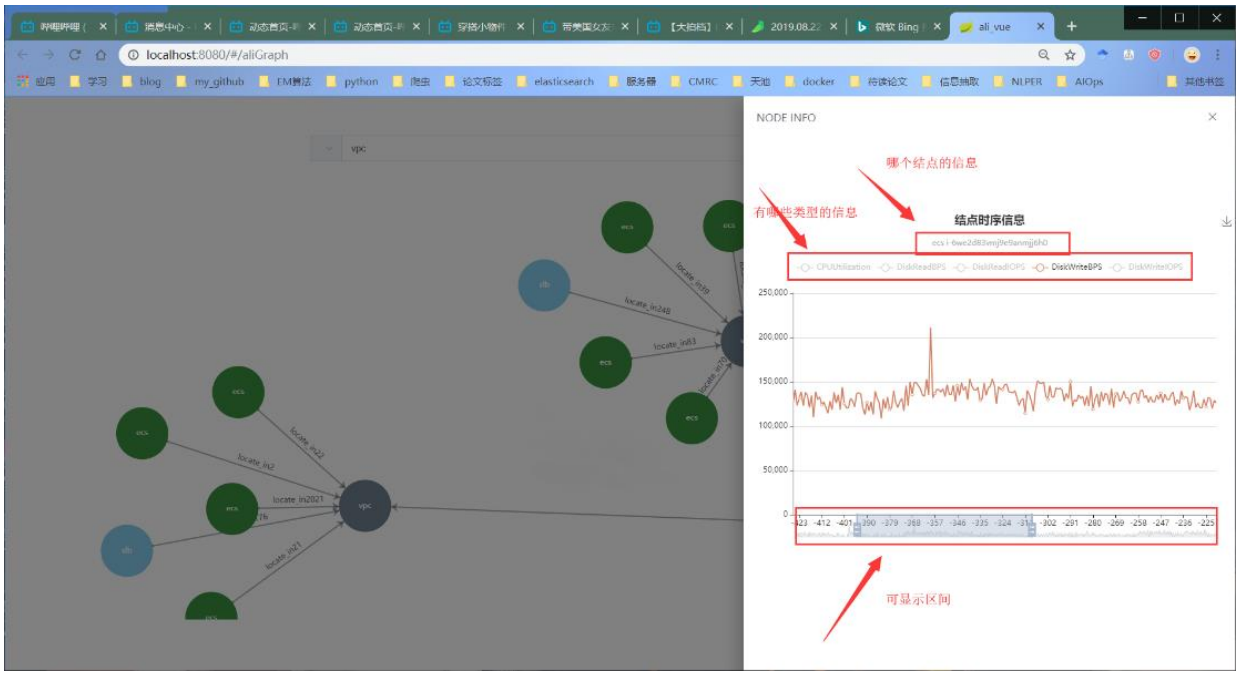

图 5-11 查询 vpc 节点的监控信息

5.4.3 故障根因分析测试

故障根因分析是本系统的核心功能，本节将会对故障根因分析及其可视化展示进行测试，测试结果如表 5-4 所示。

表 5-4 故障根因分析功能测试用例

内容项	描述
测试目的	测试故障根因分析功能
前置条件	用户进入故障根因分析首页
用例设计	1．输入查询语句“f2”，查询异常发现到的 f2 故障，点击查询按钮。 2．输入查询语句“f3”，查询异常发现到的 f3 故障，点击查询按钮。
预期结果	1．返回查询结果，查询结果是本故障的根因事件节点及其触发路径。 2．返回查询结果，查询结果是本故障的根因事件节点及其触发路径。
测试结果	两个用例经过测试与预期结果相符合
状态	通过

故障根因分析首页输入查询故障根因的界面和查询结果如图 5-12。

中国知网 https://www.cnki.net

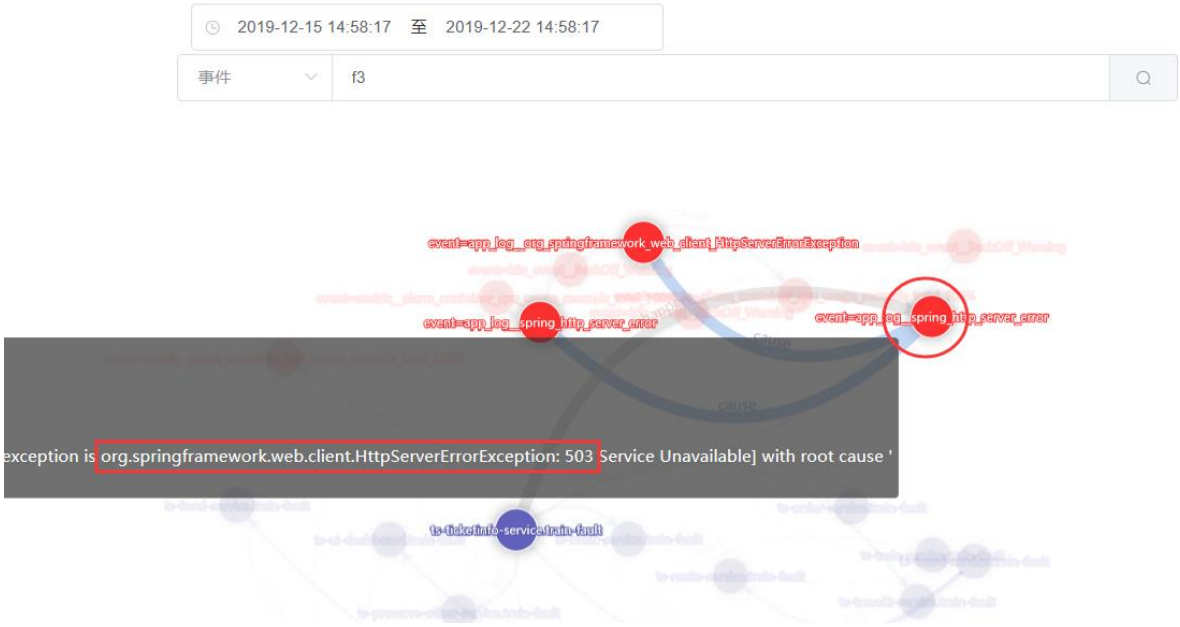

图 5-12 f3 故障的故障根因分析结果及其触发路径展示

5.4.4 系统性能测试

本文对于故障根因分析系统的性能进行测试，主要包括设备查询和故障根因分析耗时，本文采用 10 次测验的平均结果作为最终结果。结果如表 5-5 所示。

表 5-5 系统性能测试结果

内容项	耗时
模糊查询设备属性节点	1.25s
查询某类别设备节点	1.67s
查询某节点及其周边节点	1.44s
查询特定路径的节点	1.85s
查询设备监控信息	1.27s
异常发现	3.64s
故障根因分析	18.45s

5.5 本章小结

本章基于第三章和第四章提出的基于知识图谱的故障根因分析方法，设计并实现了一个故障根因分析系统。本章首先对系统进行了需求分析，然后对系统概要进行了设计，并分模块的对系统的核心功能进行详细设计并实现。最后通过系统测试对本系统进行了测试。测试表明，本系统满足故障根因分析和设备查询两个功能性需求，同时系统响应时间也在可接受范围内。

中国知网 https://www.cnki.net

第六章 总结与展望

6.1 本文工作总结

本文首先介绍了故障根因分析的研究背景与研究意义，以及现阶段故障根因分析的主要方法和挑战。本文面临的主要问题是如何沉淀历史数据中显式的知识用来提高故障根因分析的精度并完整地解释出事件层次的故障触发路径，为此本文提出了基于知识图谱的故障根因分析方法，设计并实现了一整套的故障根因分析系统，本文的主要工作有：

1) 针对集群中的故障为每类故障构建了故障知识图谱。本文通过对历史数据的挖掘，将杂乱无章的历史数据沉淀为了显式的故障知识图谱，为故障根因分析提供了高质量的结构化数据，同时进行了相关实验，测试和评估了故障知识图谱的构建方法。
2) 提出了一种结合图卷积网络和知识图谱的故障根因分析方法。本文通过图神经网络比较当前故障传播图和故障知识图谱的相似程度判断故障的根本原因和传播路径。本文设置了多组对比实验来验证本文方法的有效性，实验结果表明，本文提出的方法在不同的评价指标上均优于其它的根因分析方法。
3) 本文基于上述的故障根因分析方法设计并实现了一个故障根因分析系统。该系统可以查看故障的 top3 根因及其触发路径，并支持设备-事件两层可视化，同时提供设备查询页面，可以查看设备的属性、监控和拓扑结构。经过测试，本系统满足设计的功能和非功能需求。

6.2 未来工作展望

本文提出的故障根因分析方法相比于现有的方法虽然在精度与性能上取得了一定的提高，但是仍然存在改善空间：

1) 本文中的图相似度计算方法采用的是图卷积神经网络，此种方法更偏向于计算图拓扑结构的相似度，对于节点相似度有所欠缺。所以在今后的研究中可以根据节点事件的结构研究节点相似度，提高图相似度计算的效果。同时可以考虑采用度量学习的方法计算图相似度。
2) 本文中溯源采用的是比较实时故障传播图和故障知识图谱间的相似度完成的，忽略了传播图本身的特性。今后的研究可以着眼于研究故障传播的特性，尝试结合故障知识图谱进行传播路径的推理。
3) 本文构建的故障知识图谱的作用是辅助故障根因分析，忽略了故障知识图谱的其他功能。今后可以考虑结合故障知识图谱，研究故障预测、自动巡检和自动故障处理，挖掘故障知识图谱的能力，研究更完善更强大的智能运维体系。

中国知网 https://www.cnki.net

致 谢

光阴似箭，日月如梭，不知不觉中三年的硕士生涯就结束了，回故自己三年的硕士生涯，当中离不开老师的教导、同学的帮助和家人的鼓励。感谢你们在这三年中给我的帮助。

本文是在漆桂林教授的指导下完成的，从论文的选题、理论分析到论文的撰写，漆老师都给了我悉心的教诲和无私的帮助。在三年的研究生生涯中，漆桂林老师总能指引我前进的方向，激励我研究先进的科学，同时给了我进行科学实践的机会。在此感谢漆老师的帮助，让我在短短的三年中不仅提升了专业知识，还学以致用将所学的知识应用到工程实践中去。每个人都会犯错，漆老师总是宽容的对待犯错的同学，在我犯错的时候及时帮我解决问题，在此再次感谢漆桂林老师。

本篇文章也离不开吴天星老师和实验室其他同学的帮助，在此我要特别感谢吴天星老师对于我论文的选题、方法研究、实验设计和论文撰写上的帮助和指导。也感谢实验室其他同学给我的灵感，感谢同学和舍友的一路陪伴，是你们的关心帮我走到了今天。还要感谢李炜卓博后三年中对我的悉心指导，每次在我遇到困难的时候都能给我很大的帮助。还有感谢校外导师丁岩在实践中给与的帮助。

最后我要感谢我的父母和姐姐，感谢他们二十多年来的教诲和支持，无论什么时候都在默默支持我，你们是我最强大的后盾。还要感谢花花陪伴我走过最困难的时光，你的陪伴是我前进的动力，让我们余生共同努力！

最后感谢百忙之中参加评阅、答辩的各位专家、教授，谢谢你们给予启迪！

中国知网 https://www.cnki.net

参考文献

[1] David B. Containers and cloud: from LXC to docker to kubernetes[J]. IEEE Cloud Computing. 2014, 1(3): 81-84.

[2] Sainul K. Amazon com goes down for 45 minutes loses 5m in business. http://techcircle.vccircle.com/2013/08/20/amazon-website-goes-down-on-monday-company-loses-5m-in-business[Z]. 2013.

[3] Solé M, Muntés M V, Rana A I, et al. Survey on models and techniques for root-cause analysis[J]. Computing Research Repository. 2017.

[4] Reali G, Femminella M, Monacelli L. Probabilistic codebook-based fault localization in data networks[J]. IEEE Transactions on Network and Service Management. 2018, 15(2): 567-581.

[5] Meng Y, Zhang S, Sun Y, et al. Localizing failure root causes in a microservice through causality inference[C]. IEEE/ACM International Symposium on Quality of Service. 2020: 1-10.

[6] Brandón Á, Solé M, Huélamo A, et al. Graph-based root cause analysis for service-oriented and microservice architectures[J]. Journal of Systems and Software. 2020, 159: 110432.

[7] Nie X, Zhao Y, Sui K, et al. Mining causality graph for automatic web-based service diagnosis[C]. IEEE International Performance Computing and Communications Conference. 2016: 1-8.

[8] Klemettinen M, Mannila H, Toivonen H. rule discovery in telecommunication alarm data[J]. Journal of network and systems management. 1999, 7(4): 395-423.

[9] Liu G, Mok K A, Yang J E. Composite events for network event correlation[C]. Integrated Network Management. 1999: 247-260.

[10] Wang H Z, Nguyen P, Li J, et al. GRANO: Interactive graph-based root cause analysis for cloud-native distributed data platform[J]. Proceedings of the VLDB Endowment. 2019, 12(12): 1942-1945.

[11] Johnsson A, Meirosu C. Towards automatic network fault localization in real time using probabilistic inference[C]. IFIP/IEEE International Symposium on Integrated Network Management. 2013: 1393-1398.

[12] Johnsson A, Meirosu C, Flinta C. Online network performance degradation localization using probabilistic inference and change detection[C]. IEEE/IFIP Network Operations and Management Symposium. 2014: 1-8.

[13] Dubois D, Lang J, Prade H. Automated reasoning using possibilistic logic: semantics, belief revision, and variable certainty weights[J]. IEEE Transactions on Knowledge and Data Engineering. 1994, 6(1): 64-71.

[14] Yang Q, Yang X, Zhu X, et al. A fault diagnosis method of transmission network based on bayesian network and fault decision table[C]. 2020 5th Asia Conference on Power and Electrical Engineering. 2020: 42-46.

[15] Salfner F, Malek M. Using hidden semi-markov models for effective online failure prediction[C]. 2007 26th IEEE International Symposium on Reliable Distributed Systems. 2007: 161-174.

[16] Li S X, Hua J F, Luo Z. Survey on hidden Markov model based fault diagnosis and prognosis[C]. 2013 25th Chinese Control and Decision Conference. 2013: 4880-4884.

中国知网 https://www.cnki.net

[17] Bennacer L, Amirat Y, Chibani A, et al. Self-diagnosis technique for virtual private networks combining bayesian networks and case-based reasoning[J]. IEEE Transactions on Automation Science and Engineering. 2015, 12(1): 354-366.

[18] Diana L S, Soumaya Y. Root cause analysis of familiarity biases in classification of inventory items based on logical patterns recognition[J]. Computers & Industrial Engineering. 2016, 93: 121-130.

[19] Chen M, Zheng X A, Lloyd J, et al. Failure diagnosis using decision trees[C]. International Conference on Autonomic Computing. 2004: 36-43.

[20] Bauer M, Cox J W, Michelle H C, et al. Nearest neighbors methods for root cause analysis of plantwide disturbances[J]. Industrial & Engineering Chemistry Research. 2007, 46: 5977-5984.

[21] Klaus J. Clustering intrusion detection alarms to support root cause analysis[J]. ACM Transactions on Information and System Security. 2003, 6: 443-471.

[22] Sorsa T, Koivo H. Application of artificial neural networks in process fault diagnosis[J]. Autom. 1993, 29: 843-849.

[23] Wang L, Zhao N, Chen J, et al. Root-cause metric location for microservice systems via log anomaly detection[C]. 2020 IEEE International Conference on Web Services. 2020: 142-150.

[24] Du M, Li F F, Zheng G N, et al. DeepLog: anomaly detection and diagnosis from system logs through deep learning[J]. Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. 2017: 1285-1298.

[25] Bonatti P, Decker S, Polleres A, et al. Knowledge graphs: new directions for knowledge representation on the semantic web[J]. Dagstuhl Reports. 2018, 8: 29-111.

[26] Bizer C, Tom H, Tim B L. Linked data - the story so far[J]. International Journal on Semantic Web and Information Systems. 2009, 5: 1-22.

[27] Niu X, Sun X R, Wang H, et al. Zhishi.me - weaving chinese linking open data[C]. International Semantic Web Conference. 2011: 205-220.

[28] Jens L, Robert I, Max J, et al. DBpedia - a large-scale, multilingual knowledge base extracted from Wikipedia[J]. Semantic Web. 2015, 6: 167-195.

[29] Bollacker K, Evans C, Paritosh P, et al. Freebase: a collaboratively created graph database for structuring human knowledge[C]. SIGMOD Conference. 2008: 1247-1250.

[30] Suchanek F M, Kasneci G, Weikum G. Yago: a core of semantic knowledge[C]. International World Wide Web Conferences. 2007: 697-706.

[31] Xu Z L, Sheng Y P, He L R, et al. Review on knowledge graph techniques[J]. Journal of University of Electronic Science and Technology of China. 2016: 589-606.

[32] Wang Z, Li J Z, Wang Z C, et al. XLore: a Large-scale English-Chinese bilingual knowledge graph[C]. International Semantic Web Conference. 2013: 121-124.

[33] Xu B, Xu Y, Liang J, et al. CN-DBpedia: a never-ending Chinese knowledge extraction system[C]. International Conference on Industrial, Engineering & Other Applications of Applied Intelligent Systems.

中国知网 https://www.cnki.net

2017: 428-438.

[34] Kiesling E, Ekelhart A, Kurniawan K, et al. The SEPSES knowledge graph: an integrated resource for cybersecurity[C]. IEEE International Semantic Web Conference. 2019: 198-214.

[35] Ding Y, Luo Y, Zhuang X, et al. Simple Testing Can Prevent Most Critical Failures: An Analysis of Production Failures in Distributed Data-Intensive Systems[J]. login Usenix Mag. 2014, 40.

[36] Wong W E, Gao R, Li Y, et al. A survey on software fault localization[J]. IEEE Transactions on Software Engineering. 2016, 42(8): 707-740.

[37] Joshi K R, Hiltunen M A, Sanders W H, et al. Automatic model-driven recovery in distributed systems[C]. 24th IEEE Symposium on Reliable Distributed Systems. 2005: 25-38.

[38] Wolfgang M, Stumptner M. Model-based debugging using multiple abstract models[J]. Computing Research Repository. 2003.

[39] Foggia P, Percannella G, Vento M. Graph matching and learning in pattern recognition in the last 10 years[J]. International Journal of Pattern Recognition and Artificial Intelligence. 2014, 28.

[40] Dzena H R, Pelillo M. Metrics for attributed graphs based on the maximal similarity common subgraph[J]. International Journal of Pattern Recognition and Artificial Intelligence . 2004, 18: 299-313.

[41] Hu W H, Liu B, Joseph G, et al. Pre-training graph neural networks[J]. ArXiv. 2019: 265-273.

[42] Zhou J, Cui G Q, Zhang Z Y, et al. Graph neural networks: a review of methods and applications[J]. Computing Research Repository. 2018.

[43] Bruna J, Zaremba W, Szlam A D, et al. Spectral networks and locally connected networks on graphs[J]. Computing Research Repository. 2014.

[44] Defferrard M, Bresson X, Vandergheynst P. Convolutional neural networks on graphs with fast localized spectral filtering[C]. Conference on Neural Information Processing Systems. 2016: 3837-3845.

[45] Thomas K, Welling M. Semi-supervised classification with graph convolutional networks[C]. International Conference on Learning Representations.2017.

[46] Chen J J, Ma T F, Xiao C. FastGCN: fast learning with graph convolutional networks via importance sampling[J]. Computing Research Repository. 2018.

[47] Berg V D, Ivan T, Welling M, et al. Modeling relational data with graph convolutional networks[C]. Extended Semantic Web Conference. 2018: 593-607

[48] Guha S, Rastogi R, Kyuseok S. CURE: an efficient clustering algorithm for large databases[C]. SIGMOD Conference. 1998: 73-84.

[49] Guha S, Rastogi R, Kyuseok S. ROCK: a robust clustering algorithm for categorical attributes[J]. Proceedings 15th International Conference on Data Engineering. 1999: 512-521.

[50] Shervashidze N, Schweitzer P, Leeuwen E J V, et al. Weisfeiler-Lehman graph kernels[J]. Journal of Machine Learning Research. 2011, 12: 2539-2561.

[51] Yang L, Yang N. An integrated event summarization approach for complex system management[J]. IEEE

中国知网 https://www.cnki.net

Transactions on Network and Service Management. 2019, 16(2): 550-562.

[52] Landauer M, Skopik F, Wurzenberger M, et al. System log clustering approaches for cyber security applications: A survey[J]. Computers & Security. 2020, 92: 101739.

中国知网 https://www.cnki.net

作者攻读硕士学位期间科研成果

发表的论文

[1] Weizhuo Li, Buye Zhang, Liang Xu, Meng Wang, **Anyuan Luo** and Yan Niu. Combining Knowledge Graph Embedding and Network Embedding for Detecting Similar Mobile Applications [C]. The 9th International Conference on Natural Language Processing and Chinese Computing (NLPCC 2020), pp. 256-269. (EI Index)

待投稿的论文

[1] Tianxing Wu, Guilin Qi, **Anyuan Luo** and Sudong Fang. Reducing the Difficulty of IT Operations-an Automatic Approach to Root Cause Analysis for Distributed Systems. IEEE Transactions on Knowledge and Data Engineering. (under review)

中国知网 https://www.cnki.net

中国知网 https://www.cnki.net